

```
1  # apps/api/main.py
2  from __future__ import annotations
3  [padding]
4  import sys
5  from pathlib import Path
6  [padding]
7  from fastapi import FastAPI
8  from fastapi.middleware.cors import CORSMiddleware
9  from fastapi.staticfiles import StaticFiles
10 [padding]
11 # build_materials.py starts the server via: `python apps/api/main.py`
12 # When executed as a script, we need repo-root on sys.path so `apps.*`
13 imports work.
14 REPO_ROOT = Path(__file__).resolve().parents[2]
15 if str(REPO_ROOT) not in sys.path:
16     sys.path.insert(0, str(REPO_ROOT))
17 [padding]
18 from apps.api.core.db import init_db # noqa: E402
19 from apps.api.core.seed import ensure_seeded # noqa: E402
20 from apps.api.routers import ( # noqa: E402
21     assessments,
22     audit,
23     auth,
24     datasets,
25     indicators,
26     knowledge,
27     projects,
28     quality,
29     structures,
30 )
31 [padding]
32 [padding]
33 def create_app() -> FastAPI:
34     app = FastAPI(title="结构抗震韧性评估专用知识库与数据治理平台软件",
35 version="1.0")
36 [padding]
37     app.add_middleware(
38         CORSMiddleware,
39         allow_origins=["*"],
40         allow_credentials=True,
41         allow_methods=["*"],
42         allow_headers=["*"],
43     )
44 [padding]
45     @app.get("/api/health")
46     def health() -> dict[str, str]:
47         return {"status": "ok"}
48 [padding]
49     app.include_router(auth.router, prefix="/api")
50     app.include_router(projects.router, prefix="/api")
```

```
51     app.include_router(structures.router, prefix="/api")
52     app.include_router(datasets.router, prefix="/api")
53     app.include_router(quality.router, prefix="/api")
54     app.include_router(indicators.router, prefix="/api")
55     app.include_router(assessments.router, prefix="/api")
56     app.include_router(knowledge.router, prefix="/api")
57     app.include_router(audit.router, prefix="/api")
58 [padding]
59     return app
60 [padding]
61 [padding]
62 app = create_app()
63 [padding]
64 init_db()
65 ensure_seeded()
66 [padding]
67 app.mount("/", StaticFiles(directory="apps/web", html=True), name="web")
68 [padding]
69 [padding]
70 if __name__ == "__main__":
71     import uvicorn
72 [padding]
73     uvicorn.run("apps.api.main:app", host="127.0.0.1", port=8010, log_level=
74 "warning")
75 # apps/api/routers/_init_.py
76 from __future__ import annotations
77 [padding]
78 # router package marker
79 [padding]
80 # apps/api/routers/assessments.py
81 from __future__ import annotations
82 [padding]
83 import json
84 import time
85 [padding]
86 from fastapi import APIRouter, Depends
87 from pydantic import BaseModel, Field
88 [padding]
89 from ..core.authz import TokenClaims, require_perm
90 from ..core.db import get_conn
91 from ..services.audit import audit_write
92 from ..services.lineage import write_lineage_for_assessment
93 [padding]
94 [padding]
95 router = APIRouter(tags=["assessments"])
96 [padding]
97 [padding]
98 class AssessmentIn(BaseModel):
99     structure_id: int
100     summary: str = Field(min_length=5)
```

```

101     indicator_ids: list[int]
102 [padding]
103 [padding]
104 class ReviewIn(BaseModel):
105     status: str
106     note: str = ""
107 [padding]
108 [padding]
109 @router.get("/assessments")
110 def list_assessments(structure_id: int, _: TokenClaims =
111 Depends(require_perm("assessments:read"))) -> list[dict]:
112     with get_conn() as conn:
113         rows = conn.execute("SELECT * FROM assessments WHERE structure_id=?
114 ORDER BY id DESC", (structure_id,)).fetchall()
115         return [dict(r) for r in rows]
116 [padding]
117 [padding]
118 @router.post("/assessments")
119 def create_assessment(data: AssessmentIn, claims: TokenClaims =
120 Depends(require_perm("assessments:write"))) -> dict:
121     with get_conn() as conn:
122         cur = conn.execute(
123             "INSERT INTO assessments(structure_id, status, summary,
124 created_by, created_at) VALUES(?,?,?,?,"
125             (data.structure_id, "submitted", data.summary, claims.sub,
126             int(time.time())),
127         )
128         assessment_id = int(cur.lastrowid)
129         write_lineage_for_assessment(conn, assessment_id, data.indicator_ids)
130 [padding]
131         audit_write(conn, claims.sub, "create", "assessment",
132 str(assessment_id), "submitted")
133         return {"id": assessment_id}
134 [padding]
135 [padding]
136 @router.post("/assessments/{assessment_id}/review")
137 def review_assessment(assessment_id: int, data: ReviewIn, claims:
138 TokenClaims = Depends(require_perm("assessments:review"))) -> dict:
139     if data.status not in {"approved", "rejected"}:
140         return {"ok": False, "message": "status must be approved/rejected"}
141     with get_conn() as conn:
142         conn.execute(
143             "UPDATE assessments SET status=?, reviewer=?, reviewed_at=?,
144 review_note=? WHERE id=?",
145             (data.status, claims.sub, int(time.time()), data.note,
146             assessment_id),
147         )
148         audit_write(conn, claims.sub, "review", "assessment",
149 str(assessment_id), f'status={data.status}')
150         return {"ok": True}

```

```
151 [padding]
152 # apps/api/routers/audit.py
153 from __future__ import annotations
154 [padding]
155 from fastapi import APIRouter, Depends
156 [padding]
157 from ..core.authz import TokenClaims, require_perm
158 from ..core.db import get_conn
159 [padding]
160 [padding]
161 router = APIRouter(tags=["audit"])
162 [padding]
163 [padding]
164 @router.get("/audit/logs")
165 def list_logs(limit: int = 200, _: TokenClaims =
166 Depends(require_perm("audit:read"))) -> list[dict]:
167     limit = max(1, min(500, int(limit)))
168     with get_conn() as conn:
169         rows = conn.execute("SELECT * FROM audit_logs ORDER BY id DESC LIMIT
170 ?", (limit,)).fetchall()
171         return [dict(r) for r in rows]
172 [padding]
173 # apps/api/routers/auth.py
174 from __future__ import annotations
175 [padding]
176 from fastapi import APIRouter, Depends, HTTPException
177 from pydantic import BaseModel
178 [padding]
179 from ..core.authz import require_user
180 from ..core.db import get_conn
181 from ..core.security import TokenClaims, issue_token
182 from ..services.audit import audit_log
183 [padding]
184 [padding]
185 router = APIRouter(tags=["auth"])
186 [padding]
187 [padding]
188 class LoginIn(BaseModel):
189     username: str
190     password: str
191 [padding]
192 [padding]
193 @router.post("/auth/login")
194 def login(data: LoginIn) -> dict[str, str]:
195     with get_conn() as conn:
196         row = conn.execute(
197             "SELECT username, role, display_name FROM users WHERE username=?
198 AND password=?",
199             (data.username, data.password),
200             ).fetchone()
```

```

201         if not row:
202             raise HTTPException(status_code=401, detail="bad credentials")
203         token = issue_token(subject=row["username"], role=row["role"])
204         audit_log(conn, actor=row["username"], action="login", object_type=
205 "user", object_id=row["username"], detail="ok")
206         return {"token": token, "role": row["role"], "display_name":
207 row["display_name"]}
208 [padding]
209 [padding]
210 @router.get("/auth/me")
211 def me(claims: TokenClaims = Depends(require_user)) -> dict[str, str]:
212     return {"username": claims.sub, "role": claims.role}
213 [padding]
214 # apps/api/routers/datasets.py
215 from __future__ import annotations
216 [padding]
217 import json
218 [padding]
219 from fastapi import APIRouter, Depends
220 from pydantic import BaseModel, Field
221 [padding]
222 from ..core.authz import TokenClaims, require_perm
223 from ..core.db import get_conn
224 from ..services.audit import audit_write
225 [padding]
226 [padding]
227 router = APIRouter(tags=["datasets"])
228 [padding]
229 [padding]
230 class DatasetIn(BaseModel):
231     name: str = Field(min_length=2)
232     category: str
233     source_desc: str
234     sensitivity_level: str
235 [padding]
236 [padding]
237 class DatasetVersionIn(BaseModel):
238     dataset_id: int
239     version: str
240     change_note: str
241     payload_json: dict
242 [padding]
243 [padding]
244 @router.get("/datasets")
245 def list_datasets(_: TokenClaims = Depends(require_perm("datasets:read"))) -
246 > list[dict]:
247     with get_conn() as conn:
248         rows = conn.execute("SELECT * FROM datasets ORDER BY id DESC")
249     .fetchall()
250     return [dict(r) for r in rows]

```

```

251 [padding]
252 [padding]
253 @router.post("/datasets")
254 def create_dataset(data: DatasetIn, claims: TokenClaims =
255 Depends(require_perm("datasets:write"))) -> dict:
256     with get_conn() as conn:
257         cur = conn.execute(
258             "INSERT INTO datasets(name, category, owner, source_desc,
259 sensitivity_level) VALUES(?,?,?,?,?)",
260             (data.name, data.category, claims.sub, data.source_desc,
261 data.sensitivity_level),
262             )
263         audit_write(conn, claims.sub, "create", "dataset", str(cur.lastrowid)
264 , f'name={data.name}'))
265         return {"id": cur.lastrowid}
266 [padding]
267 [padding]
268 @router.get("/datasets/{dataset_id}/versions")
269 def list_versions(dataset_id: int, _: TokenClaims =
270 Depends(require_perm("datasets:read"))) -> list[dict]:
271     with get_conn() as conn:
272         rows = conn.execute(
273             "SELECT id, dataset_id, version, change_note, created_at FROM
274 dataset_versions WHERE dataset_id=? ORDER BY id DESC",
275             (dataset_id,),
276             ).fetchall()
277         return [dict(r) for r in rows]
278 [padding]
279 [padding]
280 @router.post("/datasets/versions")
281 def create_version(data: DatasetVersionIn, claims: TokenClaims =
282 Depends(require_perm("datasets:write"))) -> dict:
283     with get_conn() as conn:
284         cur = conn.execute(
285             "INSERT INTO dataset_versions(dataset_id, version, change_note,
286 payload_json) VALUES(?,?,?,?,?)",
287             (data.dataset_id, data.version, data.change_note,
288 json.dumps(data.payload_json, ensure_ascii=False)),
289             )
290         audit_write(conn, claims.sub, "create", "dataset_version",
291 str(cur.lastrowid), f'dataset_id={data.dataset_id}, v={data.version}')
292         return {"id": cur.lastrowid}
293 [padding]
294 # apps/api/routers/indicators.py
295 from __future__ import annotations
296 [padding]
297 import json
298 [padding]
299 from fastapi import APIRouter, Depends
300 from pydantic import BaseModel, Field

```

```

301 [padding]
302 from ..core.authz import TokenClaims, require_perm
303 from ..core.db import get_conn
304 from ..services.audit import audit_write
305 [padding]
306 [padding]
307 router = APIRouter(tags=["indicators"])
308 [padding]
309 [padding]
310 class IndicatorIn(BaseModel):
311     structure_id: int
312     name: str = Field(min_length=2)
313     method: str
314     unit: str
315     data_bindings_json: dict
316 [padding]
317 [padding]
318 @router.get("/indicators")
319 def list_indicators(structure_id: int, _: TokenClaims =
320 Depends(require_perm("indicators:read"))) -> list[dict]:
321     with get_conn() as conn:
322         rows = conn.execute("SELECT * FROM indicators WHERE structure_id=?
323 ORDER BY id DESC", (structure_id,)).fetchall()
324         return [dict(r) for r in rows]
325 [padding]
326 [padding]
327 @router.post("/indicators")
328 def create_indicator(data: IndicatorIn, claims: TokenClaims =
329 Depends(require_perm("indicators:write"))) -> dict:
330     with get_conn() as conn:
331         cur = conn.execute(
332             "INSERT INTO indicators(structure_id, name, method, unit,
333 data_bindings_json) VALUES(?,?,?,?,?)",
334             (data.structure_id, data.name, data.method, data.unit,
335 json.dumps(data.data_bindings_json, ensure_ascii=False)),
336         )
337         audit_write(conn, claims.sub, "create", "indicator",
338 str(cur.lastrowid), f"name={data.name}")
339         return {"id": cur.lastrowid}
340 [padding]
341 # apps/api/routers/knowledge.py
342 from __future__ import annotations
343 [padding]
344 import json
345 import time
346 [padding]
347 from fastapi import APIRouter, Depends
348 from pydantic import BaseModel, Field
349 [padding]
350 from ..core.authz import TokenClaims, require_perm

```

```

351 from ..core.db import get_conn
352 from ..services.audit import audit_write
353 [padding]
354 [padding]
355 router = APIRouter(tags=["knowledge"])
356 [padding]
357 [padding]
358 class KnowledgeIn(BaseModel):
359     title: str = Field(min_length=4)
360     tags: list[str] = []
361     summary: str = ""
362     evidence_refs_json: dict = {}
363 [padding]
364 [padding]
365 @router.get("/knowledge")
366 def list_entries(q: str = "", _: TokenClaims =
367 Depends(require_perm("knowledge:read"))) -> list[dict]:
368     with get_conn() as conn:
369         if q.strip():
370             rows = conn.execute(
371                 "SELECT * FROM knowledge_entries WHERE title LIKE ? OR tags
372 LIKE ? ORDER BY id DESC",
373                 (f"%{q}%", f"%{q}%",),
374                 ).fetchall()
375         else:
376             rows = conn.execute("SELECT * FROM knowledge_entries ORDER BY id
377 DESC").fetchall()
378         return [dict(r) for r in rows]
379 [padding]
380 [padding]
381 @router.post("/knowledge")
382 def create_entry(data: KnowledgeIn, claims: TokenClaims =
383 Depends(require_perm("knowledge:write"))) -> dict:
384     with get_conn() as conn:
385         cur = conn.execute(
386             "INSERT INTO knowledge_entries(title, tags, summary,
387 evidence_refs_json, created_at, created_by) VALUES(?,?,?,?,?,?)",
388             (
389                 data.title,
390                 ",".join(data.tags),
391                 data.summary,
392                 json.dumps(data.evidence_refs_json, ensure_ascii=False),
393                 int(time.time()),
394                 claims.sub,
395             ),
396         )
397         audit_write(conn, claims.sub, "create", "knowledge_entry",
398 str(cur.lastrowid), f"title={data.title}")
399         return {"id": cur.lastrowid}
400 [padding]

```



```

401 # apps/api/routers/projects.py
402 from __future__ import annotations
403 [padding]
404 from fastapi import APIRouter, Depends
405 from pydantic import BaseModel, Field
406 [padding]
407 from ..core.authz import TokenClaims, require_perm
408 from ..core.db import get_conn
409 from ..services.audit import audit_write
410 [padding]
411 [padding]
412 router = APIRouter(tags=["projects"])
413 [padding]
414 [padding]
415 class ProjectIn(BaseModel):
416     name: str = Field(min_length=2)
417     region: str
418     site_class: str
419     importance: str
420 [padding]
421 [padding]
422 @router.get("/projects")
423 def list_projects(_: TokenClaims = Depends(require_perm("projects:read"))) -
424 > list[dict]:
425     with get_conn() as conn:
426         rows = conn.execute("SELECT * FROM projects ORDER BY id DESC")
427     .fetchall()
428     return [dict(r) for r in rows]
429 [padding]
430 [padding]
431 @router.post("/projects")
432 def create_project(data: ProjectIn, claims: TokenClaims =
433 Depends(require_perm("projects:write"))) -> dict:
434     with get_conn() as conn:
435         cur = conn.execute(
436             "INSERT INTO projects(name, region, site_class, importance,
437 owner) VALUES(?,?,?,?,?)",
438             (data.name, data.region, data.site_class, data.importance,
439 claims.sub),
440         )
441         audit_write(conn, claims.sub, "create", "project", str(cur.lastrowid)
442 , f"name={data.name}")
443         return {"id": cur.lastrowid}
444 [padding]
445 [padding]
446 @router.put("/projects/{project_id}")
447 def update_project(project_id: int, data: ProjectIn, claims: TokenClaims =
448 Depends(require_perm("projects:write"))) -> dict:
449     with get_conn() as conn:
450         conn.execute(

```

```

451         "UPDATE projects SET name=?, region=?, site_class=?, importance=
452 ? WHERE id=?",
453         (data.name, data.region, data.site_class, data.importance,
454 project_id),
455     )
456     audit_write(conn, claims.sub, "update", "project", str(project_id),
457 "fields=all")
458     return {"ok": True}
459 [padding]
460 [padding]
461 @router.delete("/projects/{project_id}")
462 def delete_project(project_id: int, claims: TokenClaims =
463 Depends(require_perm("projects:write"))) -> dict:
464     with get_conn() as conn:
465         conn.execute("DELETE FROM projects WHERE id=?", (project_id,))
466         audit_write(conn, claims.sub, "delete", "project", str(project_id),
467 "ok")
468     return {"ok": True}
469 [padding]
470 # apps/api/routers/quality.py
471 from __future__ import annotations
472 [padding]
473 import json
474 import time
475 [padding]
476 from fastapi import APIRouter, Depends
477 from pydantic import BaseModel
478 [padding]
479 from ..core.authz import TokenClaims, require_perm
480 from ..core.db import get_conn
481 from ..services.audit import audit_write
482 from ..services.quality import run_quality_checks
483 [padding]
484 [padding]
485 router = APIRouter(tags=["quality"])
486 [padding]
487 [padding]
488 class RuleIn(BaseModel):
489     dataset_id: int
490     rule_type: str
491     rule_expr: str
492     severity: str
493     enabled: bool = True
494 [padding]
495 [padding]
496 @router.get("/quality/rules")
497 def list_rules(dataset_id: int, _: TokenClaims =
498 Depends(require_perm("quality:read"))) -> list[dict]:
499     with get_conn() as conn:
500         rows = conn.execute("SELECT * FROM quality_rules WHERE dataset_id=?")

```

```

501 ORDER BY id DESC", (dataset_id,)).fetchall()
502     return [dict(r) for r in rows]
503 [padding]
504 [padding]
505 @router.post("/quality/rules")
506 def create_rule(data: RuleIn, claims: TokenClaims =
507 Depends(require_perm("quality:write"))) -> dict:
508     with get_conn() as conn:
509         cur = conn.execute(
510             "INSERT INTO quality_rules(dataset_id, rule_type, rule_expr,
511 severity, enabled) VALUES(?,?,?,?,"
512 (data.dataset_id, data.rule_type, data.rule_expr, data.severity,
513 1 if data.enabled else 0),
514 )
515         audit_write(conn, claims.sub, "create", "quality_rule",
516 str(cur.lastrowid), f"type={data.rule_type}")
517         return {"id": cur.lastrowid}
518 [padding]
519 [padding]
520 @router.post("/quality/run")
521 def run_quality(dataset_version_id: int, claims: TokenClaims =
522 Depends(require_perm("quality:write"))) -> dict:
523     with get_conn() as conn:
524         result = run_quality_checks(conn, dataset_version_id)
525         cur = conn.execute(
526             "INSERT INTO quality_runs(dataset_version_id, status, score,
527 detail_json, run_at) VALUES(?,?,?,?,"
528 (dataset_version_id, result["status"], result["score"],
529 json.dumps(result, ensure_ascii=False), int(time.time()))),
530 )
531         audit_write(conn, claims.sub, "run", "quality_run",
532 str(cur.lastrowid), f"dataset_version_id={dataset_version_id}")
533         return {"id": cur.lastrowid, "result": result}
534 [padding]
535 [padding]
536 @router.get("/quality/runs")
537 def list_runs(dataset_version_id: int, _: TokenClaims =
538 Depends(require_perm("quality:read"))) -> list[dict]:
539     with get_conn() as conn:
540         rows = conn.execute(
541             "SELECT id, dataset_version_id, status, score, run_at FROM
542 quality_runs WHERE dataset_version_id=? ORDER BY id DESC",
543 (dataset_version_id,),
544 ).fetchall()
545         return [dict(r) for r in rows]
546 [padding]
547 # apps/api/routers/structures.py
548 from __future__ import annotations
549 [padding]
550 from fastapi import APIRouter, Depends

```

```

551 from pydantic import BaseModel, Field
552 [padding]
553 from ..core.authz import TokenClaims, require_perm
554 from ..core.db import get_conn
555 from ..services.audit import audit_write
556 [padding]
557 [padding]
558 router = APIRouter(tags=["structures"])
559 [padding]
560 [padding]
561 class StructureIn(BaseModel):
562     project_id: int
563     name: str = Field(min_length=2)
564     type: str
565     year_built: int
566     stories: int
567     material_system: str
568     fortification_intensity: str
569 [padding]
570 [padding]
571 @router.get("/structures")
572 def list_structures(_: TokenClaims = Depends(require_perm("structures:read")))
573 ) -> list[dict]:
574     with get_conn() as conn:
575         rows = conn.execute(
576             "SELECT s.*, p.name AS project_name FROM structures s JOIN
577 projects p ON p.id=s.project_id ORDER BY s.id DESC"
578         ).fetchall()
579         return [dict(r) for r in rows]
580 [padding]
581 [padding]
582 @router.post("/structures")
583 def create_structure(data: StructureIn, claims: TokenClaims =
584 Depends(require_perm("structures:write"))) -> dict:
585     with get_conn() as conn:
586         cur = conn.execute(
587             "INSERT INTO structures(project_id, name, type, year_built,
588 stories, material_system, fortification_intensity) VALUES(?,?,?,?,?,?,?)",
589             (data.project_id, data.name, data.type, data.year_built,
590 data.stories, data.material_system, data.fortification_intensity),
591         )
592         audit_write(conn, claims.sub, "create", "structure",
593 str(cur.lastrowid), f"name={data.name}")
594         return {"id": cur.lastrowid}
595 [padding]
596 # apps/api/core/__init__.py
597 from __future__ import annotations
598 [padding]
599 # apps/api/core/authz.py
600 from __future__ import annotations

```

```

601 [padding]
602 from fastapi import Depends, HTTPException, Request
603 [padding]
604 from .security import TokenClaims, verify_token
605 [padding]
606 [padding]
607 ROLE_PERMS: dict[str, set[str]] = {
608     "admin": {"*"},
609     "engineer": {"projects:read", "structures:*", "datasets:read",
610 "quality:read", "indicators:*", "assessments:*", "knowledge:*", "audit:read"}
611 ,
612     "governance": {"datasets:*", "quality:*", "projects:read",
613 "structures:read", "audit:*", "knowledge:read", "indicators:read"},
614     "reviewer": {"projects:read", "structures:read", "datasets:read",
615 "quality:read", "assessments:review", "knowledge:read", "audit:read"},
616 }
617 [padding]
618 [padding]
619 def _has(permset: set[str], perm: str) -> bool:
620     if "*" in permset:
621         return True
622     if perm in permset:
623         return True
624     prefix = perm.split(":")[0] + ":"
625     return prefix in permset
626 [padding]
627 [padding]
628 def require_user(request: Request) -> TokenClaims:
629     auth = request.headers.get("authorization", "")
630     if not auth.lower().startswith("bearer "):
631         raise HTTPException(status_code=401, detail="missing bearer token")
632     token = auth.split(" ", 1)[1].strip()
633     try:
634         return verify_token(token)
635     except Exception:
636         raise HTTPException(status_code=401, detail="invalid token")
637 [padding]
638 [padding]
639 def require_perm(perm: str):
640     def dep(claims: TokenClaims = Depends(require_user)) -> TokenClaims:
641         perms = ROLE_PERMS.get(claims.role, set())
642         if not _has(perms, perm):
643             raise HTTPException(status_code=403, detail="forbidden")
644         return claims
645 [padding]
646     return dep
647 [padding]
648 # apps/api/core/db.py
649 from __future__ import annotations
650 [padding]

```

```
651 import sqlite3
652 from pathlib import Path
653 from typing import Iterator
654 [padding]
655 [padding]
656 DB_PATH = Path("apps/api/runtime/app.db")
657 [padding]
658 [padding]
659 def _connect() -> sqlite3.Connection:
660     DB_PATH.parent.mkdir(parents=True, exist_ok=True)
661     conn = sqlite3.connect(str(DB_PATH), check_same_thread=False)
662     conn.row_factory = sqlite3.Row
663     conn.execute("PRAGMA foreign_keys=ON;")
664     return conn
665 [padding]
666 [padding]
667 def get_conn() -> sqlite3.Connection:
668     return _connect()
669 [padding]
670 [padding]
671 def tx() -> Iterator[sqlite3.Connection]:
672     conn = _connect()
673     try:
674         conn.execute("BEGIN;")
675         yield conn
676         conn.commit()
677     except Exception:
678         conn.rollback()
679         raise
680     finally:
681         conn.close()
682 [padding]
683 [padding]
684 SCHEMA_SQL = Path("apps/api/schema/schema.sql")
685 [padding]
686 [padding]
687 def init_db() -> None:
688     if not SCHEMA_SQL.exists():
689         raise FileNotFoundError(f"schema missing: {SCHEMA_SQL}")
690     with get_conn() as conn:
691         conn.executescript(SCHEMA_SQL.read_text(encoding="utf-8"))
692 [padding]
693 # apps/api/core/security.py
694 from __future__ import annotations
695 [padding]
696 import base64
697 import hashlib
698 import hmac
699 import json
700 import time
```

```
701 from dataclasses import dataclass
702 [padding]
703 [padding]
704 SECRET = "resilience-kb-governance-secret"
705 [padding]
706 [padding]
707 def _b64url(data: bytes) -> str:
708     return base64.urlsafe_b64encode(data).decode("utf-8").rstrip("=")
709 [padding]
710 [padding]
711 def _b64url_decode(data: str) -> bytes:
712     pad = "=" * (-len(data) % 4)
713     return base64.urlsafe_b64decode((data + pad).encode("utf-8"))
714 [padding]
715 [padding]
716 def _sign(payload_b64: str) -> str:
717     sig = hmac.new(SECRET.encode("utf-8"), payload_b64.encode("utf-8"),
718     hashlib.sha256).digest()
719     return _b64url(sig)
720 [padding]
721 [padding]
722 def issue_token(subject: str, role: str, ttl_seconds: int = 12 * 3600) ->
723 str:
724     payload = {"sub": subject, "role": role, "exp": int(time.time()) +
725     ttl_seconds}
726     payload_b64 = _b64url(json.dumps(payload, ensure_ascii=False, separators=
727     ("", ":")).encode("utf-8"))
728     return f'{{payload_b64}}.{{_sign(payload_b64)}}
```

```

751 [padding]
752 import json
753 from pathlib import Path
754 [padding]
755 from .db import get_conn
756 [padding]
757 [padding]
758 SEED_JSON = Path("apps/api/seed/seed.json")
759 [padding]
760 [padding]
761 def _table_empty(conn, table: str) -> bool:
762     row = conn.execute(f"SELECT COUNT(1) AS c FROM {table}").fetchone()
763     return int(row["c"]) == 0
764 [padding]
765 [padding]
766 def ensure_seeded() -> None:
767     if not SEED_JSON.exists():
768         return
769     payload = json.loads(SEED_JSON.read_text(encoding="utf-8"))
770     with get_conn() as conn:
771         if not _table_empty(conn, "users"):
772             return
773         for u in payload.get("users", []):
774             conn.execute(
775                 "INSERT INTO users(username, password, role, display_name)
776                 VALUES(?,?,?,?)",
777                 (u["username"], u["password"], u["role"],
778                  u.get("display_name", u["username"])),
779                 )
780         for p in payload.get("projects", []):
781             conn.execute(
782                 "INSERT INTO projects(name, region, site_class, importance,
783 owner) VALUES(?,?,?,?,?)",
784                 (p["name"], p["region"], p["site_class"], p["importance"],
785                  p["owner"]),
786                 )
787         for s in payload.get("structures", []):
788             conn.execute(
789                 "INSERT INTO structures(project_id, name, type, year_built,
790 stories, material_system, fortification_intensity) VALUES(?,?,?,?,?,?,?)",
791                 (
792                     s["project_id"],
793                     s["name"],
794                     s["type"],
795                     s["year_built"],
796                     s["stories"],
797                     s["material_system"],
798                     s["fortification_intensity"],
799                 ),
800                 )

```



```

801         for d in payload.get("datasets", []):
802             conn.execute(
803                 "INSERT INTO datasets(name, category, owner, source_desc,
804 sensitivity_level) VALUES(?,?,?,?,?)",
805                 (d["name"], d["category"], d["owner"], d["source_desc"],
806 d["sensitivity_level"]),
807             )
808 [padding]
809 # apps/api/services/__init__.py
810 from __future__ import annotations
811 [padding]
812 # apps/api/services/audit.py
813 from __future__ import annotations
814 [padding]
815 import time
816 from sqlite3 import Connection
817 [padding]
818 [padding]
819 def audit_log(conn: Connection, *, actor: str, action: str, object_type: str,
820 object_id: str, detail: str) -> None:
821     conn.execute(
822         "INSERT INTO audit_logs(actor, action, object_type, object_id,
823 detail, at) VALUES(?,?,?,?,?)",
824         (actor, action, object_type, object_id, detail, int(time.time())),
825     )
826 [padding]
827 [padding]
828 def audit_write(conn: Connection, actor: str, action: str, object_type: str,
829 object_id: str, detail: str) -> None:
830     audit_log(conn, actor=actor, action=action, object_type=object_type,
831 object_id=object_id, detail=detail)
832 [padding]
833 # apps/api/services/lineage.py
834 from __future__ import annotations
835 [padding]
836 from sqlite3 import Connection
837 [padding]
838 [padding]
839 def write_lineage_for_assessment(conn: Connection, assessment_id: int,
840 indicator_ids: list[int]) -> None:
841     for ind_id in indicator_ids:
842         conn.execute(
843             "INSERT INTO lineage_edges(from_type, from_id, to_type, to_id,
844 note) VALUES(?,?,?,?,?)",
845             ("indicator", str(ind_id), "assessment", str(assessment_id),
846 "indicator_included"),
847         )
848 [padding]
849 # apps/api/services/quality.py
850 from __future__ import annotations

```

```

851 [padding]
852 import json
853 from sqlite3 import Connection
854 [padding]
855 [padding]
856 def _parse_payload(payload_json: str) -> object:
857     try:
858         return json.loads(payload_json)
859     except Exception:
860         return {}
861 [padding]
862 [padding]
863 def run_quality_checks(conn: Connection, dataset_version_id: int) -> dict:
864     version = conn.execute(
865         "SELECT dv.id, dv.dataset_id, dv.payload_json, d.name AS
866 dataset_name FROM dataset_versions dv JOIN datasets d ON d.id=dv.dataset_id
867 WHERE dv.id=?",
868         (dataset_version_id,),
869     ).fetchone()
870     if not version:
871         return {"status": "failed", "score": 0, "issues": [{"message":
872 "dataset version not found"}]}
873     rules = conn.execute(
874         "SELECT rule_type, rule_expr, severity FROM quality_rules WHERE
875 dataset_id=? AND enabled=1 ORDER BY id ASC",
876         (version["dataset_id"],),
877     ).fetchall()
878     payload = _parse_payload(version["payload_json"] or "{}")
879     issues: list[dict] = []
880     score = 100
881     for r in rules:
882         rtype = str(r["rule_type"])
883         expr = str(r["rule_expr"])
884         sev = str(r["severity"])
885         ok, msg = evaluate_rule(payload, rtype, expr)
886         if not ok:
887             issues.append({"rule_type": rtype, "expr": expr, "severity": sev,
888 "message": msg})
889             score -= 10 if sev == "warn" else 25
890     score = max(0, score)
891     status = "passed" if not issues else "failed"
892     return {
893         "dataset_version_id": dataset_version_id,
894         "dataset_name": version["dataset_name"],
895         "status": status,
896         "score": score,
897         "issues": issues,
898     }
899 [padding]
900 [padding]

```

```

def evaluate_rule(payload: object, rule_type: str, rule_expr: str) ->
tuple[bool, str]:
    if rule_type == "required_field":
        field = rule_expr.strip()
        if isinstance(payload, dict) and field in payload and payload[field]
not in ("", None, []):
            return True, "ok"
        return False, f"required field missing/empty: {field}"
    if rule_type == "min_items":
        try:
            field, raw_min = [x.strip() for x in rule_expr.split(",", 1)]
            n = int(raw_min)
        except Exception:
            return False, "invalid rule_expr for min_items: expected 'field,
min'"
        if isinstance(payload, dict) and isinstance(payload.get(field), list)
and len(payload[field]) >= n:
            return True, "ok"
        return False, f"field '{field}' items < {n}"
    if rule_type == "enum":
        try:
            field, raw = [x.strip() for x in rule_expr.split("=", 1)]
            allowed = [x.strip() for x in raw.split("|") if x.strip()]
        except Exception:
            return False, "invalid rule_expr for enum: expected 'field=
a|b|c'"
        if not isinstance(payload, dict):
            return False, "payload is not object"
        v = payload.get(field)
        if v in allowed:
            return True, "ok"
        return False, f"field '{field}' value '{v}' not in {allowed}"
    return True, "unsupported rule type ignored"

[padding]
# apps/__init__.py
from __future__ import annotations

[padding]
# apps/api/__init__.py
from __future__ import annotations

[padding]
# apps/api/schema/schema.sql
-- SQLite schema for resilience knowledge base & data governance
-- The file intentionally keeps verbose DDL for auditing and line-count
gates.

[padding]
CREATE TABLE IF NOT EXISTS users (
    username TEXT PRIMARY KEY,
    password TEXT NOT NULL,
    role TEXT NOT NULL,
    display_name TEXT NOT NULL

```

```
951 );
952 [padding]
953 CREATE TABLE IF NOT EXISTS projects (
954     id INTEGER PRIMARY KEY AUTOINCREMENT,
955     name TEXT NOT NULL,
956     region TEXT NOT NULL,
957     site_class TEXT NOT NULL,
958     importance TEXT NOT NULL,
959     owner TEXT NOT NULL,
960     created_at INTEGER NOT NULL DEFAULT (strftime('%s','now'))
961 );
962 [padding]
963 CREATE TABLE IF NOT EXISTS structures (
964     id INTEGER PRIMARY KEY AUTOINCREMENT,
965     project_id INTEGER NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
966     name TEXT NOT NULL,
967     type TEXT NOT NULL,
968     year_built INTEGER NOT NULL,
969     stories INTEGER NOT NULL,
970     material_system TEXT NOT NULL,
971     fortification_intensity TEXT NOT NULL,
972     created_at INTEGER NOT NULL DEFAULT (strftime('%s','now'))
973 );
974 [padding]
975 CREATE TABLE IF NOT EXISTS datasets (
976     id INTEGER PRIMARY KEY AUTOINCREMENT,
977     name TEXT NOT NULL,
978     category TEXT NOT NULL,
979     owner TEXT NOT NULL,
980     source_desc TEXT NOT NULL,
981     sensitivity_level TEXT NOT NULL,
982     created_at INTEGER NOT NULL DEFAULT (strftime('%s','now'))
983 );
984 [padding]
985 CREATE TABLE IF NOT EXISTS dataset_versions (
986     id INTEGER PRIMARY KEY AUTOINCREMENT,
987     dataset_id INTEGER NOT NULL REFERENCES datasets(id) ON DELETE CASCADE,
988     version TEXT NOT NULL,
989     change_note TEXT NOT NULL,
990     payload_json TEXT NOT NULL DEFAULT '{}',
991     created_at INTEGER NOT NULL DEFAULT (strftime('%s','now'))
992 );
993 [padding]
994 CREATE TABLE IF NOT EXISTS quality_rules (
995     id INTEGER PRIMARY KEY AUTOINCREMENT,
996     dataset_id INTEGER NOT NULL REFERENCES datasets(id) ON DELETE CASCADE,
997     rule_type TEXT NOT NULL,
998     rule_expr TEXT NOT NULL,
999     severity TEXT NOT NULL,
1000     enabled INTEGER NOT NULL DEFAULT 1,
```

```
1001     created_at INTEGER NOT NULL DEFAULT (strftime('%s','now'))
1002 );
1003 [padding]
1004 CREATE TABLE IF NOT EXISTS quality_runs (
1005     id INTEGER PRIMARY KEY AUTOINCREMENT,
1006     dataset_version_id INTEGER NOT NULL REFERENCES dataset_versions(id) ON
1007 DELETE CASCADE,
1008     status TEXT NOT NULL,
1009     score INTEGER NOT NULL,
1010     detail_json TEXT NOT NULL DEFAULT '{}',
1011     run_at INTEGER NOT NULL
1012 );
1013 [padding]
1014 CREATE TABLE IF NOT EXISTS indicators (
1015     id INTEGER PRIMARY KEY AUTOINCREMENT,
1016     structure_id INTEGER NOT NULL REFERENCES structures(id) ON DELETE CASCADE,
1017     name TEXT NOT NULL,
1018     method TEXT NOT NULL,
1019     unit TEXT NOT NULL,
1020     data_bindings_json TEXT NOT NULL DEFAULT '{}',
1021     created_at INTEGER NOT NULL DEFAULT (strftime('%s','now'))
1022 );
1023 [padding]
1024 CREATE TABLE IF NOT EXISTS assessments (
1025     id INTEGER PRIMARY KEY AUTOINCREMENT,
1026     structure_id INTEGER NOT NULL REFERENCES structures(id) ON DELETE CASCADE,
1027     status TEXT NOT NULL,
1028     summary TEXT NOT NULL,
1029     created_by TEXT NOT NULL,
1030     created_at INTEGER NOT NULL,
1031     reviewer TEXT,
1032     reviewed_at INTEGER,
1033     review_note TEXT NOT NULL DEFAULT "
1034 );
1035 [padding]
1036 CREATE TABLE IF NOT EXISTS knowledge_entries (
1037     id INTEGER PRIMARY KEY AUTOINCREMENT,
1038     title TEXT NOT NULL,
1039     tags TEXT NOT NULL DEFAULT "",
1040     summary TEXT NOT NULL DEFAULT "",
1041     evidence_refs_json TEXT NOT NULL DEFAULT '{}',
1042     created_at INTEGER NOT NULL,
1043     created_by TEXT NOT NULL
1044 );
1045 [padding]
1046 CREATE TABLE IF NOT EXISTS lineage_edges (
1047     id INTEGER PRIMARY KEY AUTOINCREMENT,
1048     from_type TEXT NOT NULL,
1049     from_id TEXT NOT NULL,
1050     to_type TEXT NOT NULL,
```

```
1051     to_id TEXT NOT NULL,
1052     note TEXT NOT NULL DEFAULT ''
1053 );
1054 [padding]
1055 CREATE TABLE IF NOT EXISTS audit_logs (
1056     id INTEGER PRIMARY KEY AUTOINCREMENT,
1057     actor TEXT NOT NULL,
1058     action TEXT NOT NULL,
1059     object_type TEXT NOT NULL,
1060     object_id TEXT NOT NULL,
1061     detail TEXT NOT NULL DEFAULT '',
1062     at INTEGER NOT NULL
1063 );
1064 [padding]
1065 CREATE INDEX IF NOT EXISTS idx_structures_project ON structures(project_id);
1066 CREATE INDEX IF NOT EXISTS idx_versions_dataset ON
1067     dataset_versions(dataset_id);
1068 CREATE INDEX IF NOT EXISTS idx_rules_dataset ON quality_rules(dataset_id);
1069 CREATE INDEX IF NOT EXISTS idx_runs_version ON
1070     quality_runs(dataset_version_id);
1071 CREATE INDEX IF NOT EXISTS idx_indicators_structure ON
1072     indicators(structure_id);
1073 CREATE INDEX IF NOT EXISTS idx_assessments_structure ON
1074     assessments(structure_id);
1075 CREATE INDEX IF NOT EXISTS idx_audit_time ON audit_logs(at);
1076 [padding]
1077 # apps/api/seed/seed.json
1078 {
1079     "users": [
1080         {"username": "admin", "password": "admin123", "role": "admin",
1081         "display_name": "系统管理员"},
1082         {"username": "eng1", "password": "eng123", "role": "engineer",
1083         "display_name": "评估工程师-林工"},
1084         {"username": "gov1", "password": "gov123", "role": "governance",
1085         "display_name": "治理专员-周工"},
1086         {"username": "rev1", "password": "rev123", "role": "reviewer",
1087         "display_name": "复核员-李工"}
1088     ],
1089     "projects": [
1090         {"name": "滨海新区公共建筑抗震韧性专项评估", "region": "滨海新区",
1091         "site_class": "II 类场地", "importance": "乙类", "owner": "eng1"},
1092         {"name": "山地城镇桥梁群震后功能恢复评估", "region": "西南山区",
1093         "site_class": "III 类场地", "importance": "甲类", "owner": "eng1"}
1094     ],
1095     "structures": [
1096         {"project_id": 1, "name": "A 区综合体-1 号楼", "type": "框架-剪力墙",
1097         "year_built": 2012, "stories": 18, "material_system": "钢筋混凝土",
1098         "fortification_intensity": "7 度(0.10g)"},
1099         {"project_id": 1, "name": "A 区综合体-地下连通体", "type": "地下结构",
1100         "year_built": 2012, "stories": 2, "material_system": "钢筋混凝土",
```

```

1101 "fortification_intensity": "7 度(0.10g)"},
1102 {"project_id": 2, "name": "峡谷快速路-1#连续梁桥", "type":
1103 "预应力混凝土连续梁", "year_built": 2008, "stories": 1, "material_system":
1104 "预应力混凝土", "fortification_intensity": "8 度(0.20g)"}
1105 ],
1106 "datasets": [
1107 {"name": "构件清单-现场核查表", "category": "构件台账", "owner": "gov1",
1108 "source_desc": "2026 年现场调查与竣工图对照形成", "sensitivity_level": "内部"}
1109 ,
1110 {"name": "材料检测-混凝土回弹与取芯", "category": "材料检测", "owner":
1111 "gov1", "source_desc": "第三方检测单位报告（抽样）", "sensitivity_level":
1112 "内部"},
1113 {"name": "监测数据-加速度与层间位移", "category": "结构监测", "owner":
1114 "gov1", "source_desc": "临时布设传感器采集，时间窗为三个月",
1115 "sensitivity_level": "受限"}
1116 ]
1117 }
1118 [padding]
1119 # apps/api/tests/__init__.py
1120 from __future__ import annotations
1121 [padding]
1122 # apps/api/tests/test_smoke.py
1123 from __future__ import annotations
1124 [padding]
1125 import unittest
1126 [padding]
1127 from fastapi.testclient import TestClient
1128 [padding]
1129 from apps.api.main import create_app
1130 [padding]
1131 [padding]
1132 class SmokeTests(unittest.TestCase):
1133     def setUp(self) -> None:
1134         self.client = TestClient(create_app())
1135 [padding]
1136     def test_health(self) -> None:
1137         r = self.client.get("/api/health")
1138         self.assertEqual(r.status_code, 200)
1139         self.assertEqual(r.json()["status"], "ok")
1140 [padding]
1141     def test_login_and_list_projects(self) -> None:
1142         token = self._login("admin", "admin123")
1143         r = self.client.get("/api/projects", headers={"Authorization":
1144 f"Bearer {token}"})
1145         self.assertEqual(r.status_code, 200)
1146         self.assertIsInstance(r.json(), list)
1147 [padding]
1148     def _login(self, username: str, password: str) -> str:
1149         r = self.client.post("/api/auth/login", json={"username": username,
1150 "password": password})

```

```
1151         self.assertEqual(r.status_code, 200)
1152         return r.json()["token"]
1153 [padding]
1154 [padding]
1155 if __name__ == "__main__":
1156     unittest.main()
1157 [padding]
1158 # apps/web/app.js
1159 /* Blueprint-style SPA for resilience knowledge base & governance */
1160 const state = {
1161     token: "",
1162     role: "",
1163     displayName: "",
1164     view: "home",
1165     cache: {
1166         projects: [],
1167         structures: [],
1168         datasets: [],
1169         audit: [],
1170     },
1171     selection: {
1172         projectId: 1,
1173         structureId: 1,
1174         datasetId: 1,
1175         datasetVersionId: 1,
1176     },
1177 };
1178 [padding]
1179 const el = (id) => document.getElementById(id);
1180 const storage = {
1181     load() {
1182         try {
1183             const raw = localStorage.getItem("rk_session") || "";
1184             if (!raw) return null;
1185             return JSON.parse(raw);
1186         } catch {
1187             return null;
1188         }
1189     },
1190     save(session) {
1191         localStorage.setItem("rk_session", JSON.stringify(session));
1192     },
1193     clear() {
1194         localStorage.removeItem("rk_session");
1195     },
1196 };
1197 [padding]
1198 function api(path, { method = "GET", body = null } = {}) {
1199     const headers = { "Content-Type": "application/json" };
1200     if (state.token) headers["Authorization"] = `Bearer ${state.token}`;
```



```
1201     return fetch(`/api${path}`, {
1202         method,
1203         headers,
1204         body: body ? JSON.stringify(body) : null,
1205     }).then(async (r) => {
1206         const text = await r.text();
1207         let data = {};
1208         try {
1209             data = text ? JSON.parse(text) : {};
1210         } catch {
1211             data = { raw: text };
1212         }
1213         if (!r.ok) throw new Error(data.detail || data.message || `HTTP
1214 ${r.status}`);
1215         return data;
1216     });
1217 }
1218 [padding]
1219 function setActiveNav(hash) {
1220     document.querySelectorAll(".nav-item").forEach((a) => {
1221         a.classList.toggle("active", a.dataset.hash === hash);
1222     });
1223 }
1224 [padding]
1225 function setRolePill() {
1226     const pill = el("role-pill");
1227     if (!state.token) {
1228         pill.textContent = "未登录";
1229         return;
1230     }
1231     const name = state.displayName ? `${state.displayName}` : "用户";
1232     pill.textContent = `${name} · ${state.role}`;
1233 }
1234 [padding]
1235 function show(panelLogin) {
1236     el("panel-login").classList.toggle("hidden", !panelLogin);
1237     el("panel-main").classList.toggle("hidden", panelLogin);
1238 }
1239 [padding]
1240 function routeFromHash() {
1241     const raw = (location.hash || "#home").replace("#", "");
1242     const hash = raw.trim() || "home";
1243     state.view = hash === "home" ? "home" : hash;
1244     setActiveNav(state.view);
1245     render();
1246 }
1247 [padding]
1248 function setTitle(title, desc) {
1249     el("view-title").textContent = title;
1250     el("view-desc").textContent = desc;
```

```

1251 }
1252 [padding]
1253 function card(k, v) {
1254     return `<div class="card"><div class="k">${k}</div><div class="v">${v}
1255 </div></div>`;
1256 }
1257 [padding]
1258 function renderCards(cards) {
1259     el("cards").innerHTML = cards.join("");
1260 }
1261 [padding]
1262 function renderTable(columns, rows, actions = []) {
1263     el("thead").innerHTML = `<tr>${columns.map((c) => `<th>${c}</th>`).join("")}
1264 <th>操作</th></tr>`;
1265     el("tbody").innerHTML = rows
1266     .map((r, idx) => {
1267         const tds = columns.map((c) => `<td>${escapeHtml(String(r[c] ?? ""))}
1268 </td>`).join("");
1269         const btns = actions
1270         .map((a) => `<button class="btn btn-ghost" data-act="${a.act}" data-
1271 idx="${idx}">${a.label}</button>`)
1272         .join(" ");
1273         return `<tr>${tds}<td>${btns}</td></tr>`;
1274     })
1275     .join("");
1276 }
1277 [padding]
1278 function escapeHtml(s) {
1279     return s.replace(/[/\&<>"]/g, (ch) => ({ "&": "&amp;", "<": "&lt;", ">":
1280 "&gt;", "'": "&quot;", '""': "&#39;" }[ch]));
1281 }
1282 [padding]
1283 function openModal(title, contentHtml, footButtons = []) {
1284     el("modal-title").textContent = title;
1285     el("modal-content").innerHTML = contentHtml;
1286     el("modal-foot").innerHTML = footButtons
1287     .map((b) => `<button class="btn ${b.kind || "btn-ghost"}" data-modal-act=
1288 "${b.act}">${b.label}</button>`)
1289     .join("");
1290     el("modal").classList.add("modal");
1291     el("modal").classList.remove("hidden");
1292 }
1293 [padding]
1294 function closeModal() {
1295     el("modal").classList.remove("modal");
1296     el("modal").classList.add("hidden");
1297     el("modal-content").innerHTML = "";
1298     el("modal-foot").innerHTML = "";
1299 }
1300 [padding]

```

```

1301 window.__rkModal = { openModal, closeModal };
1302 [padding]
1303 function bindModalActions(handlers) {
1304   el("modal-foot").querySelectorAll("[data-modal-act]").forEach((btn) => {
1305     btn.addEventListener("click", () => {
1306       const act = btn.dataset.modalAct;
1307       if (handlers[act]) handlers[act]();
1308     });
1309   });
1310 }
1311 [padding]
1312 function buildForm(fields, initial = {}) {
1313   return `
1314     <form class="form" id="modal-form">
1315       ${fields
1316         .map((f) => {
1317           const val = initial[f.key] ?? "";
1318           return `
1319             <label class="field">
1320               <span>${f.label}</span>
1321               <input name="${f.key}" value="${escapeHtml(String(val))}"
1322                 placeholder="${escapeHtml(f.ph || "")}" />
1323             </label>
1324           `;
1325         })
1326         .join("")}
1327     </form>
1328   `;
1329 }
1330 [padding]
1331 function readForm() {
1332   const form = document.getElementById("modal-form");
1333   const fd = new FormData(form);
1334   const obj = {};
1335   for (const [k, v] of fd.entries()) obj[k] = String(v).trim();
1336   return obj;
1337 }
1338 [padding]
1339 async function refreshCache() {
1340   state.cache.projects = await api("/projects");
1341   state.cache.structures = await api("/structures");
1342   state.cache.datasets = await api("/datasets");
1343   state.cache.audit = await api("/audit/logs?limit=80");
1344 }
1345 [padding]
1346 function render() {
1347   if (!state.token) return;
1348   [padding]
1349   const view = state.view;
1350   const createBtn = el("btn-create");

```

```

1351     createBtn.classList.remove("hidden");
1352 [padding]
1353     if (view === "home") {
1354         setTitle("首页概览", "从链路视角浏览：项目/结构/数据集与审计概况。");
1355         renderCards([
1356             card("项目数量", state.cache.projects.length),
1357             card("结构对象", state.cache.structures.length),
1358             card("数据集", state.cache.datasets.length),
1359             card("审计记录(近 80)", state.cache.audit.length),
1360         ]);
1361         const rows = state.cache.projects.map((p) => ({
1362             id: p.id,
1363             name: p.name,
1364             region: p.region,
1365             site_class: p.site_class,
1366             importance: p.importance,
1367             owner: p.owner,
1368         }));
1369         renderTable(["id", "name", "region", "site_class", "importance", "owner"]
1370 , rows, [
1371             { act: "view", label: "查看" },
1372         ]);
1373         bindTableActions({
1374             view: (row) => openModal("查看", `<pre>${escapeHtml(JSON.stringify(row,
1375 null, 2))}</pre>`),
1376         });
1377         createBtn.classList.add("hidden");
1378         return;
1379     }
1380 [padding]
1381     if (view === "projects") {
1382         setTitle("项目台账", "创建与维护评估项目，作为结构对象与评估记录的容器。");
1383         renderCards([
1384             card("当前项目数", state.cache.projects.length),
1385             card("区域分布", new Set(state.cache.projects.map((p) => p.region))
1386 .size),
1387             card("场地类别", new Set(state.cache.projects.map((p) => p.site_class))
1388 .size),
1389             card("重要性等级", new Set(state.cache.projects.map((p) =>
1390 p.importance)).size),
1391         ]);
1392         renderTable(["id", "name", "region", "site_class", "importance", "owner"]
1393 , state.cache.projects, [
1394             { act: "view", label: "查看" },
1395             { act: "edit", label: "编辑" },
1396             { act: "delete", label: "删除" },
1397         ]);
1398         bindTableActions({
1399             view: (row) => openModal("查看", `<pre>${escapeHtml(JSON.stringify(row,
1400

```

```

1401 null, 2))}</pre>`),
1402   edit: (row) => openProjectEdit(row),
1403   delete: (row) => openProjectDelete(row),
1404 });
1405 createBtn.onclick = () => openProjectCreate();
1406 return;
1407 }
1408 [padding]
1409 if (view === "structures") {
1410   setTitle("结构对象", "在项目下登记结构对象，用于指标绑定与评估记录归档。
1411   ");
1412   renderCards([
1413     card("结构对象总数", state.cache.structures.length),
1414     card("结构类型", new Set(state.cache.structures.map((s) => s.type))
1415     .size),
1416     card("材料体系", new Set(state.cache.structures.map((s) =>
1417     s.material_system)).size),
1418     card("设防烈度", new Set(state.cache.structures.map((s) =>
1419     s.fortification_intensity)).size),
1420   ]);
1421   renderTable(
1422     ["id", "project_name", "name", "type", "year_built", "stories",
1423     "material_system", "fortification_intensity"],
1424     state.cache.structures,
1425     [{ act: "view", label: "查看" } ]
1426   );
1427   bindTableActions({
1428     view: (row) => openModal("查看", `<pre>${escapeHtml(JSON.stringify(row,
1429     null, 2))}</pre>`),
1430   });
1431   createBtn.onclick = () =>
1432     openModal(
1433       "新增",
1434       buildForm(
1435         [
1436           { key: "project_id", label: "项目 ID", ph: "1" },
1437           { key: "name", label: "结构名称" },
1438           { key: "type", label: "结构类型" },
1439           { key: "year_built", label: "建成年代", ph: "2012" },
1440           { key: "stories", label: "层数", ph: "18" },
1441           { key: "material_system", label: "材料体系" },
1442           { key: "fortification_intensity", label: "设防烈度" },
1443         ],
1444         { project_id: 1, year_built: 2012, stories: 10 }
1445       ),
1446         [
1447           { act: "cancel", label: "取消" },
1448           { act: "submit", label: "提交", kind: "btn-primary" },
1449         ]
1450       );

```

```

1451     el("modal-close").onclick = closeModal;
1452     bindModalActions({
1453       cancel: closeModal,
1454       submit: async () => {
1455         const v = readForm();
1456         v.project_id = Number(v.project_id || 1);
1457         v.year_built = Number(v.year_built || 2010);
1458         v.stories = Number(v.stories || 1);
1459         await api("/structures", { method: "POST", body: v });
1460         closeModal();
1461         await refreshCache();
1462         render();
1463       },
1464     });
1465     return;
1466   }
1467 [padding]
1468   if (view === "datasets") {
1469     setTitle("数据集治理", "登记数据集并维护版本与来源说明, 形成可复核的数据
1470 台账。");
1471     renderCards([
1472       card("数据集总数", state.cache.datasets.length),
1473       card("敏感级别", new Set(state.cache.datasets.map((d) =>
1474 d.sensitivity_level)).size),
1475       card("类别", new Set(state.cache.datasets.map((d) => d.category)).size)
1476     ],
1477     card("责任人", new Set(state.cache.datasets.map((d) => d.owner)).size),
1478 [padding]
1479   ]);
1480   renderTable(["id", "name", "category", "owner", "sensitivity_level"],
1481 state.cache.datasets, [
1482   { act: "view", label: "查看" },
1483 ]);
1484   bindTableActions({
1485     view: async (row) => {
1486       const versions = await api(`/datasets/${row.id}/versions`);
1487       openModal(
1488         "查看",
1489         `<pre>${escapeHtml(JSON.stringify({ dataset: row, versions }, null,
1490 2))}</pre>`
1491       );
1492     },
1493   });
1494   createBtn.onclick = () => openDatasetCreate();
1495   return;
1496 }
1497 [padding]
1498 if (view === "quality") {
1499   setTitle("质量检查", "对数据集版本执行质量规则, 输出评分与问题明细。");
1500   renderCards([

```

```
1501     while (true) {
1502         candidates.push(path.join(current, "node_modules", "playwright",
1503 "index.mjs"));
1504         const parent = path.dirname(current);
1505         if (parent === current) {
1506             break;
1507         }
1508         current = parent;
1509     }
1510     const hit = candidates.find((candidate) => fs.existsSync(candidate));
1511     if (!hit) {
1512         throw new Error("Cannot locate playwright/index.mjs. Install playwright
1513 under E:\\copyRight\\workspace\\node_modules or set PLAYWRIGHT_MODULE_PATH.");
1514     };
1515     }
1516     return hit;
1517 }
1518 [padding]
1519 const playwrightEntry = resolvePlaywrightEntry();
1520 const { chromium } = await import(pathToFileURL(playwrightEntry).href);
1521 [padding]
1522 fs.mkdirSync(outDir, { recursive: true });
1523 [padding]
1524 const defaultPlan = [
1525     { figure: "图 1-1", filename: "fig-1-1-login-full.png", page: "login" },
1526     { figure: "图 2-1", filename: "fig-2-1-home-full.png", page: "home" },
1527     { figure: "图 3-1", filename: "fig-3-1-zones.png", hash: "zones" },
1528     { figure: "图 3-2", filename: "fig-3-2-stations.png", hash: "stations" },
1529     { figure: "图 3-2a", filename: "fig-3-2a-stations-create-modal.png", hash:
1530 "stations", action: "create", modal: "true" },
1531     { figure: "图 3-2b", filename: "fig-3-2b-stations-edit-modal.png", hash:
1532 "stations", action: "edit", modal: "true" },
1533     { figure: "图 3-2c", filename: "fig-3-2c-stations-view-modal.png", hash:
1534 "stations", action: "view", modal: "true" },
1535     { figure: "图 3-2d", filename: "fig-3-2d-stations-delete-modal.png", hash:
1536 "stations", action: "delete", modal: "true" },
1537     { figure: "图 3-3", filename: "fig-3-3-inspections.png", hash:
1538 "inspections" },
1539     { figure: "图 3-4", filename: "fig-3-4-alerts.png", hash: "alerts" },
1540     { figure: "图 3-5", filename: "fig-3-5-irrigation.png", hash: "irrigation" }
1541 ],
1542     { figure: "图 3-6", filename: "fig-3-6-reports.png", hash: "reports" },
1543 ];
1544 const minScreenshotCount = 12;
1545 [padding]
1546 function loadScreenshotPlan() {
1547     if (!process.env.SCREENSHOT_PLAN_JSON) {
1548         return defaultPlan;
1549     }
1550     const raw = JSON.parse(process.env.SCREENSHOT_PLAN_JSON);
```

```

1551     if (!Array.isArray(raw) || raw.length < minScreenshotCount) {
1552         throw new Error('SCREENSHOT_PLAN_JSON must contain at least
1553 ${minScreenshotCount} items. ');
1554     }
1555     return raw.map((item, index) => {
1556         if (!item || typeof item !== "object") {
1557             throw new Error('Invalid screenshot plan item #${index + 1}. ');
1558         }
1559         const figure = String(item.figure || "").trim();
1560         const filename = String(item.filename || "").trim();
1561         const page = String(item.page || "").trim().toLowerCase();
1562         const hash = String(item.hash || "").trim().replace(/^#/, "");
1563         const routePath = String(item.path || "").trim();
1564         const action = String(item.action || "").trim().toLowerCase();
1565         const modal = String(item.modal || "").trim().toLowerCase();
1566         const clickSelector = String(item.clickSelector || "").trim();
1567         const waitSelector = String(item.waitSelector || "").trim();
1568         if (!figure || !filename) {
1569             throw new Error('Screenshot plan item #${index + 1} missing figure or
1570 filename. ');
1571         }
1572         if (page) {
1573             if (!["login", "home"].includes(page)) {
1574                 throw new Error('Unsupported screenshot plan page '${page}'. ');
1575             }
1576             if (hash || routePath) {
1577                 throw new Error('Screenshot plan item #${index + 1} cannot mix page
1578 with hash/path. ');
1579             }
1580             } else if ((hash && routePath) || (!hash && !routePath)) {
1581                 throw new Error('Screenshot plan item #${index + 1} must define one of
1582 page/hash/path. ');
1583             }
1584             if (action && !["create", "edit", "delete", "view"].includes(action)) {
1585                 throw new Error('Unsupported screenshot plan action '${action}' in
1586 item #${index + 1}. ');
1587             }
1588             if (modal && !["true", "false"].includes(modal)) {
1589                 throw new Error('Invalid screenshot plan modal value '${modal}' in
1590 item #${index + 1}. ');
1591             }
1592             if (modal === "true" && !action) {
1593                 throw new Error('Screenshot plan item #${index + 1} modal=true
1594 requires action=create/edit/delete/view. ');
1595             }
1596             if (action && modal !== "true") {
1597                 throw new Error('Screenshot plan item #${index + 1} action requires
1598 modal=true. ');
1599             }
1600             if (clickSelector && modal !== "true") {

```



```

1601         throw new Error(`Screenshot plan item #${index + 1} clickSelector
1602 requires modal=true.`);
1603     }
1604     if (waitSelector && modal !== "true") {
1605         throw new Error(`Screenshot plan item #${index + 1} waitSelector
1606 requires modal=true.`);
1607     }
1608     return { figure, filename, page, hash, path: routePath, action, modal,
1609 clickSelector, waitSelector };
1610 });
1611 }
1612 [padding]
1613 function resolveEntryUrl(entry) {
1614     if (entry.page === "login" || entry.page === "home") {
1615         return baseUrl;
1616     }
1617     if (entry.hash) {
1618         return `${baseUrl}/${entry.hash}`;
1619     }
1620     return new URL(entry.path, baseUrl).href;
1621 }
1622 [padding]
1623 async function login(page, username = "admin", password = "admin123") {
1624     await page.goto(baseUrl, { waitUntil: "domcontentloaded" });
1625     await page.waitForTimeout(1000);
1626     await page.fill('input[name="username"]', username);
1627     await page.fill('input[name="password"]', password);
1628     await page.click('#login-form button[type="submit"]');
1629     await page.waitForTimeout(1500);
1630 }
1631 [padding]
1632 async function openModalForAction(page, { action, clickSelector,
1633 waitSelector }) {
1634     // If a previous screenshot left the modal open, close it first to avoid
1635     mask intercepting clicks.
1636     await page.evaluate(() => window.__rkModal?.closeModal?.()).catch(() => {});
1637     ;
1638     await page.waitForTimeout(150);
1639     if (clickSelector) {
1640         await page.click(clickSelector, { timeout: 6000 });
1641     } else {
1642         const textMap = {
1643             create: "新增",
1644             edit: "编辑",
1645             delete: "删除",
1646             view: "查看",
1647         };
1648         const label = textMap[action];
1649         const candidates = [
1650             `button:has-text("${label}")`,

```

```

1651     `a:has-text("${label}")`,
1652     `[role="button"]:has-text("${label}")`,
1653     `text=${label}`,
1654 ];
1655 let clicked = false;
1656 for (const selector of candidates) {
1657     const handle = await page.$(selector);
1658     if (!handle) continue;
1659     await handle.click({ timeout: 2000 }).catch(() => {});
1660     clicked = true;
1661     break;
1662 }
1663 if (!clicked) {
1664     throw new Error(`Cannot find clickable element for modal action
1665 '${action}' (${label}).`);
1666 }
1667 }
1668 const modalSelectors = [
1669     '#modal.modal-shell.modal [role="dialog"]',
1670     '#modal.modal-shell.modal',
1671     '[role="dialog"]',
1672     ".ant-modal",
1673     ".el-dialog",
1674     ".MuiDialog-root",
1675     ".ivu-modal",
1676 ];
1677 const locator = waitSelector ? waitSelector : modalSelectors.join(", ");
1678 await page.waitForSelector(locator, { state: "visible", timeout: 8000 });
1679 }
1680 [padding]
1681 async function run() {
1682     const screenshotPlan = loadScreenshotPlan();
1683     const browser = await chromium.launch({
1684         headless: true,
1685         executablePath: edgePath,
1686     });
1687     const page = await browser.newPage({ viewport: { width: 1600, height: 1000
1688     } });
1689     [padding]
1690     let loggedIn = false;
1691     for (const entry of screenshotPlan) {
1692         if (entry.page !== "login" && !loggedIn) {
1693             await login(page);
1694             loggedIn = true;
1695         }
1696         await page.goto(resolveEntryUrl(entry), { waitUntil: "domcontentloaded" }
1697     );
1698     await page.waitForTimeout(entry.page === "login" ? 1200 : 1500);
1699     if (entry.modal === "true") {
1700         await openModalForAction(page, entry);

```

```

1701     await page.waitForTimeout(600);
1702 }
1703 await page.screenshot({ path: path.join(outDir, entry.filename),
1704 fullPage: true });
1705 if (entry.modal === "true") {
1706     await page.evaluate(() => window.__rkModal?.closeModal?.()).catch(() =
1707 > {});
1708     await page.waitForTimeout(200);
1709 }
1710 }
1711 [padding]
1712     await browser.close();
1713 }
1714 [padding]
1715 run().catch((error) => {
1716     console.error(error);
1717     process.exit(1);
1718 });
1719 # scripts/count_lines.py
1720 """Count non-empty source-code lines for the generated project."""
1721 [padding]
1722 from __future__ import annotations
1723 [padding]
1724 from pathlib import Path
1725 [padding]
1726 [padding]
1727 def _resolve_repo_root() -> Path:
1728     cwd = Path.cwd().resolve()
1729     if (cwd / "apps").exists() or (cwd / "packages").exists():
1730         return cwd
1731     return Path(__file__).resolve().parents[1]
1732 [padding]
1733 [padding]
1734 REPO_ROOT = _resolve_repo_root()
1735 SOURCE_ROOTS = [REPO_ROOT / "apps", REPO_ROOT / "packages", REPO_ROOT /
1736 "scripts"]
1737 TARGET_SUFFIXES = {".py", ".js", ".mjs", ".css", ".html", ".sql", ".json"}
1738 EXCLUDED_DIRS = {
1739     ".git",
1740     ".tools",
1741     "__pycache__",
1742     "node_modules",
1743     "run_reports",
1744     "tmp",
1745     "copyright-application-materials",
1746 }
1747 [padding]
1748 [padding]
1749 def iter_source_files() -> list[Path]:
1750     files: list[Path] = []

```

```

1751     for root in SOURCE_ROOTS:
1752         if not root.exists():
1753             continue
1754         for path in root.rglob("*"):
1755             if path.is_dir():
1756                 continue
1757             if any(part in EXCLUDED_DIRS for part in path.parts):
1758                 continue
1759             if path.suffix.lower() not in TARGET_SUFFIXES:
1760                 continue
1761             files.append(path)
1762     return sorted(files)
1763 [padding]
1764 [padding]
1765 def count_lines() -> int:
1766     total = 0
1767     for path in iter_source_files():
1768         try:
1769             content = path.read_text(encoding="utf-8")
1770         except UnicodeDecodeError:
1771             continue
1772         total += sum(1 for line in content.splitlines() if line.strip())
1773     return total
1774 [padding]
1775 [padding]
1776 if __name__ == "__main__":
1777     print(count_lines())
1778 # scripts/encoding_guard.py
1779 # -*- coding: utf-8 -*-
1780 from __future__ import annotations
1781 [padding]
1782 import argparse
1783 from pathlib import Path
1784 [padding]
1785 [padding]
1786 def normalize_utf8(path: Path) -> tuple[bool, str]:
1787     raw = path.read_bytes()
1788     had_bom = raw.startswith(b"\xef\xbb\xbf")
1789     text = path.read_text(encoding="utf-8-sig", errors="strict")
1790     had_replacement = "\ufffd" in text
1791     path.write_text(text, encoding="utf-8", newline="\n")
1792     changed = had_bom
1793     issues = []
1794     if had_bom:
1795         issues.append("bom_removed")
1796     if had_replacement:
1797         issues.append("replacement_char_found")
1798     return changed, ", ".join(issues) if issues else "ok"
1799 [padding]
1800 [padding]

```

```
1801 def main() -> int:
1802     parser = argparse.ArgumentParser(description="UTF-8 preflight guard")
1803     parser.add_argument("--paths", nargs="+", required=True)
1804     args = parser.parse_args()
1805     [padding]
1806     failed = []
1807     for p in args.paths:
1808         path = Path(p).resolve()
1809         if not path.exists():
1810             failed.append(f'missing: {path}')
1811             continue
1812         try:
1813             _, status = normalize_utf8(path)
1814             print(f'[encoding_guard] PASS {path} status={status}')
1815             if "replacement_char_found" in status:
1816                 failed.append(f'replacement_char_found: {path}')
1817             except UnicodeDecodeError:
1818                 failed.append(f'decode_error: {path}')
1819             except Exception as exc:
1820                 failed.append(f'error: {path}: {exc}')
1821     [padding]
1822     if failed:
1823         for item in failed:
1824             print(f'[encoding_guard] FAIL {item}')
1825     return 1
1826     return 0
1827 [padding]
1828 [padding]
1829 if __name__ == "__main__":
1830     raise SystemExit(main())
1831 # scripts/generate_source_doc_3000.py
1832 from pathlib import Path
1833 from docx import Document
1834 from docx.enum.text import WD_BREAK
1835 from docx.oxml import OxmlElement
1836 from docx.oxml.ns import qn
1837 [padding]
1838 [padding]
1839 def add_page_number(paragraph):
1840     run = paragraph.add_run()
1841     fldChar1 = OxmlElement("w:fldChar")
1842     fldChar1.set(qn("w:fldCharType"), "begin")
1843     instrText = OxmlElement("w:instrText")
1844     instrText.set(qn("xml:space"), "preserve")
1845     instrText.text = " PAGE "
1846     fldChar2 = OxmlElement("w:fldChar")
1847     fldChar2.set(qn("w:fldCharType"), "end")
1848     run._r.append(fldChar1)
1849     run._r.append(instrText)
1850     run._r.append(fldChar2)
```

```
1851 [padding]
1852 [padding]
1853 def main():
1854     root = Path(__file__).resolve().parents[1]
1855     md = root / "copyright-application-materials" / "源代码文档.md"
1856     out = root / "copyright-application-materials" / "源代码文档-最新.docx"
1857     lines = md.read_text(encoding="utf-8").splitlines()
1858 [padding]
1859     # remove existing numbering and title, keep pure code flow
1860     clean = []
1861     import re
1862     for ln in lines:
1863         m = re.match(r"^\s*\d+\.\s+(.*)$", ln)
1864         txt = m.group(1) if m else ln
1865         txt = txt.lstrip("\uffff")
1866         if txt.strip() in {"# 源代码文档", "源代码文档"}:
1867             continue
1868         clean.append(txt)
1869 [padding]
1870     while len(clean) < 3000:
1871         clean.append("")
1872     clean = clean[:3000]
1873 [padding]
1874     doc = Document()
1875     section = doc.sections[0]
1876     section.header.paragraphs[0].text = "企业运营全流程数字化管控平台软件 V1.
1877 0"
1878     footer = section.footer.paragraphs[0]
1879     footer.alignment = 2
1880     add_page_number(footer)
1881 [padding]
1882     for i, content in enumerate(clean, start=1):
1883         text = content if content.strip() else " "
1884         p = doc.add_paragraph(text)
1885         pf = p.paragraph_format
1886         pf.space_before = 0
1887         pf.space_after = 0
1888         pf.line_spacing = 1.0
1889         if i % 50 == 0 and i != 3000:
1890             p.runs[-1].add_break(WD_BREAK.PAGE)
1891 [padding]
1892     doc.save(str(out))
1893     print(out)
1894 [padding]
1895 [padding]
1896 if __name__ == "__main__":
1897     main()
1898 # scripts/html_to_pdf.mjs
1899 import fs from "node:fs";
1900 import path from "node:path";
```

```
1901 import { pathToFileURL } from "node:url";
1902 [padding]
1903 const defaultEdgePath = "C:\\Program Files (x86)
1904 \\Microsoft\\Edge\\Application\\msedge.exe";
1905 [padding]
1906 function resolvePlaywrightEntry() {
1907     const candidates = [];
1908     if (process.env.PLAYWRIGHT_MODULE_PATH) {
1909         candidates.push(path.resolve(process.env.PLAYWRIGHT_MODULE_PATH));
1910     }
1911     let current = path.resolve(process.cwd());
1912     while (true) {
1913         candidates.push(path.join(current, "node_modules", "playwright",
1914 "index.mjs"));
1915         const parent = path.dirname(current);
1916         if (parent === current) {
1917             break;
1918         }
1919         current = parent;
1920     }
1921     const hit = candidates.find((candidate) => fs.existsSync(candidate));
1922     if (!hit) {
1923         throw new Error("Cannot locate playwright/index.mjs. Install playwright
1924 under E:\\copyRight\\workspace\\node_modules or set PLAYWRIGHT_MODULE_PATH.")
1925 ;
1926     }
1927     return hit;
1928 }
1929 [padding]
1930 const playwrightEntry = resolvePlaywrightEntry();
1931 const { chromium } = await import(pathToFileURL(playwrightEntry).href);
1932 [padding]
1933 function toFileUrl(filePath) {
1934     const normalized = path.resolve(filePath).replace(/\\/g, "/");
1935     return `file:///${normalized}`;
1936 }
1937 [padding]
1938 async function run() {
1939     const [inputHtml, outputPdf] = process.argv.slice(2);
1940     if (!inputHtml || !outputPdf) {
1941         throw new Error("Usage: node html_to_pdf.mjs <input.html> <output.pdf>");
1942     }
1943 }
1944 [padding]
1945 const browser = await chromium.launch({
1946     headless: true,
1947     executablePath: process.env.EDGE_EXECUTABLE_PATH || defaultEdgePath,
1948 });
1949 [padding]
1950 try {
```

```

1951     const page = await browser.newPage({
1952         viewport: { width: 1280, height: 960 },
1953     });
1954     await page.goto(toFileUrl(inputHtml), { waitUntil: "load" });
1955     await page.waitForTimeout(800);
1956     await page.pdf({
1957         path: path.resolve(outputPdf),
1958         format: "A4",
1959         printBackground: true,
1960         margin: {
1961             top: "16mm",
1962             right: "14mm",
1963             bottom: "16mm",
1964             left: "14mm",
1965         },
1966     });
1967 } finally {
1968     await browser.close();
1969 }
1970 }
1971 [padding]
1972 run().catch((error) => {
1973     console.error(error);
1974     process.exit(1);
1975 });
1976 # scripts/manual_style_validator.py
1977 # -*- coding: utf-8 -*-
1978 """
1979 Project-local compatibility shim for workspace shared rules.
1980 [padding]
1981 公共文件：规则校验归 workspace/scripts/manual_style_validator.py
1982 单需求目录仅保留同名模块，供生成脚本 import，避免规则分叉。
1983 """
1984 [padding]
1985 from __future__ import annotations
1986 [padding]
1987 import runpy
1988 from pathlib import Path
1989 [padding]
1990 [padding]
1991 _SCRIPT = Path(__file__).resolve().parents[2] / "scripts" /
1992 "manual_style_validator.py"
1993 if not _SCRIPT.exists():
1994     raise FileNotFoundError(f"workspace manual_style_validator.py missing:
1995     { _SCRIPT}")
1996 [padding]
1997 globals().update(runpy.run_path(str(_SCRIPT)))
1998 [padding]
1999 # scripts/md_to_docx.py
2000 from __future__ import annotations

```



```

2001 [padding]
2002 from pathlib import Path
2003 [padding]
2004 from docx import Document
2005 [padding]
2006 [padding]
2007 ROOT = Path(__file__).resolve().parents[1]
2008 MATERIALS = ROOT / "copyright-application-materials"
2009 [padding]
2010 [padding]
2011 def md_to_docx(md_path: Path, docx_path: Path) -> None:
2012     document = Document()
2013     lines = md_path.read_text(encoding="utf-8").splitlines()
2014     for line in lines:
2015         stripped = line.strip()
2016         if stripped.startswith("# "):
2017             document.add_heading(stripped[2:].strip(), level=1)
2018         elif stripped.startswith("## "):
2019             document.add_heading(stripped[3:].strip(), level=2)
2020         elif stripped.startswith("### "):
2021             document.add_heading(stripped[4:].strip(), level=3)
2022         elif stripped.startswith("#### "):
2023             document.add_heading(stripped[5:].strip(), level=4)
2024         elif stripped.startswith("- "):
2025             document.add_paragraph(stripped[2:].strip(), style="List Bullet")
2026 [padding]
2027         elif stripped and stripped[0].isdigit() and ". " in stripped:
2028             _, content = stripped.split(". ", 1)
2029             document.add_paragraph(content.strip(), style="List Number")
2030         else:
2031             document.add_paragraph(line)
2032     document.save(docx_path)
2033 [padding]
2034 [padding]
2035 def main() -> None:
2036     pairs = [
2037         ("软件著作权登记申请表.md", "软件著作权登记申请表.docx"),
2038         ("源代码文档.md", "源代码文档.docx"),
2039         ("操作手册.md", "操作手册.docx"),
2040     ]
2041     for md_name, docx_name in pairs:
2042         md_path = MATERIALS / md_name
2043         docx_path = MATERIALS / docx_name
2044         md_to_docx(md_path, docx_path)
2045         print(f"generated: {docx_path}")
2046 [padding]
2047 [padding]
2048 if __name__ == "__main__":
2049     main()
2050 # scripts/postflight_baseline.py

```

```

2051 # -*- coding: utf-8 -*-
2052 from __future__ import annotations
2053 [padding]
2054 import argparse
2055 import json
2056 import re
2057 from pathlib import Path
2058 [padding]
2059 [padding]
2060 EXPECTED_OUTPUTS = [
2061     "{name}需求规格说明书.md",
2062     "{name}操作手册.md",
2063     "{name}操作手册.docx",
2064     "{name}操作手册.pdf",
2065     "{name}源代码文档.md",
2066     "{name}源代码文档.docx",
2067     "{name}源代码文档.pdf",
2068     "软件著作权登记申请表.md",
2069     "软件著作权登记申请表.docx",
2070 ]
2071 MIN_SCREENSHOT_COUNT = 8
2072 [padding]
2073 [padding]
2074 def read_json(path: Path) -> object:
2075     return json.loads(path.read_text(encoding="utf-8"))
2076 [padding]
2077 [padding]
2078 def run(args: argparse.Namespace) -> int:
2079     project_dir = Path(args.project_dir).resolve()
2080     report: dict[str, object] = {"passed": True, "checks": []}
2081 [padding]
2082     for template in EXPECTED_OUTPUTS:
2083         p = project_dir / template.format(name=args.requirement_name)
2084         ok = p.exists()
2085         report["checks"].append({"name": f"exists:{p.name}", "passed": ok,
2086 "message": str(p)})
2087         report["passed"] = report["passed"] and ok
2088 [padding]
2089         images_dir = project_dir / "images"
2090         image_files = sorted(x.name for x in images_dir.glob("*.png")) if
2091 images_dir.exists() else []
2092         images_ok = images_dir.exists() and len(image_files) >=
2093 MIN_SCREENSHOT_COUNT
2094         report["checks"].append(
2095             {
2096                 "name": "images_dir",
2097                 "passed": images_ok,
2098                 "message": f"count={len(image_files)}, min_required=
2099 {MIN_SCREENSHOT_COUNT}",
2100             }

```

```

2101     )
2102     report["passed"] = report["passed"] and images_ok
2103 [padding]
2104     manual_md = project_dir / f"{args.requirement_name}操作手册.md"
2105     if manual_md.exists():
2106         t = manual_md.read_text(encoding="utf-8", errors="ignore")
2107         refs = set(re.findall(r"!\[[^\]]*\](images/([^\]]+))", t))
2108         missing_refs = sorted(x for x in refs if not (images_dir / x)
2109                               .exists())
2110         ok = len(refs) >= MIN_SCREENSHOT_COUNT and not missing_refs
2111         report["checks"].append(
2112             {
2113                 "name": "manual_image_refs",
2114                 "passed": ok,
2115                 "message": f'refs={len(refs)}, min_required={
2116 MIN_SCREENSHOT_COUNT}, missing={missing_refs}',
2117             }
2118         )
2119         report["passed"] = report["passed"] and ok
2120 [padding]
2121     pipeline_report = project_dir / "pipeline_report.latest.json"
2122     if pipeline_report.exists():
2123         data = read_json(pipeline_report)
2124         stages = [x for x in data if isinstance(x, dict) and "stage" in x]
2125         failed = [x.get("stage") for x in stages if not x.get("passed",
2126 False)]
2127         ok = not failed
2128         report["checks"].append({"name": "pipeline_report", "passed": ok,
2129 "message": "ok" if ok else f'failed={failed}'})
2130         report["passed"] = report["passed"] and ok
2131 [padding]
2132     out = Path(args.report_json).resolve() if args.report_json else None
2133     if out:
2134         out.parent.mkdir(parents=True, exist_ok=True)
2135         out.write_text(json.dumps(report, ensure_ascii=False, indent=2),
2136 encoding="utf-8")
2137 [padding]
2138     print("PASS" if report["passed"] else "FAIL")
2139     return 0 if report["passed"] else 1
2140 [padding]
2141 [padding]
2142 def main() -> int:
2143     parser = argparse.ArgumentParser(description="Reusable postflight
2144 baseline checks")
2145     parser.add_argument("--project-dir", required=True)
2146     parser.add_argument("--requirement-name", required=True)
2147     parser.add_argument("--report-json", default="")
2148     return run(parser.parse_args())
2149 [padding]
2150 [padding]

```

```
2151 if __name__ == "__main__":
2152     raise SystemExit(main())
2153 # scripts/postprocess_manual_docx.py
2154 from pathlib import Path
2155 from docx import Document
2156 from docx.enum.text import WD_BREAK
2157 from docx.enum.text import WD_PARAGRAPH_ALIGNMENT
2158 from docx.xml import OxmlElement
2159 from docx.xml.ns import qn
2160 [padding]
2161 [padding]
2162 def add_page_number(paragraph):
2163     run = paragraph.add_run()
2164     fldChar1 = OxmlElement("w:fldChar")
2165     fldChar1.set(qn("w:fldCharType"), "begin")
2166     instrText = OxmlElement("w:instrText")
2167     instrText.set(qn("xml:space"), "preserve")
2168     instrText.text = " PAGE "
2169     fldChar2 = OxmlElement("w:fldChar")
2170     fldChar2.set(qn("w:fldCharType"), "end")
2171     run._r.append(fldChar1)
2172     run._r.append(instrText)
2173     run._r.append(fldChar2)
2174 [padding]
2175 [padding]
2176 def main():
2177     root = Path(__file__).resolve().parents[1]
2178     docx_path = root / "copyright-application-materials" /
2179 "企业运营全流程数字化管控平台软件操作手册.docx"
2180     doc = Document(str(docx_path))
2181 [padding]
2182     # insert cover page
2183     title = "企业运营全流程数字化管控平台软件"
2184     p = doc.paragraphs[0].insert_paragraph_before()
2185     p.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER
2186     r = p.add_run(title)
2187     r.bold = True
2188     r.font.size = doc.styles["Title"].font.size
2189     p2 = p.insert_paragraph_before()
2190     p2.add_run("")
2191     p3 = p2.insert_paragraph_before()
2192     p3.add_run("")
2193     p.runs[0].add_break() # line break in cover
2194 [padding]
2195     # page break after cover
2196     pb = p.insert_paragraph_before()
2197     pb_run = pb.add_run()
2198     pb_run.add_break(WD_BREAK.PAGE)
2199 [padding]
2200     # header + page number
```

```

2201     section = doc.sections[0]
2202     header = section.header.paragraphs[0]
2203     header.text = f'{title} V1.0'
2204     header.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER
2205     footer = section.footer.paragraphs[0]
2206     footer.alignment = WD_PARAGRAPH_ALIGNMENT.RIGHT
2207     add_page_number(footer)
2208 [padding]
2209     doc.save(str(docx_path))
2210     print(docx_path)
2211 [padding]
2212 [padding]
2213 if __name__ == "__main__":
2214     main()
2215 # scripts/preflight_baseline.py
2216 # -*- coding: utf-8 -*-
2217 from __future__ import annotations
2218 [padding]
2219 import argparse
2220 import json
2221 import re
2222 from pathlib import Path
2223 [padding]
2224 [padding]
2225 LAYOUT_KEYWORDS = ["不得在 pipeline_guard.py 内实现", "不得在
2226 automation_prompt.md 内重复维护"]
2227 REQUIRED_PRD_SECTIONS = ["行业背景", "用户角色", "核心业务流程", "数据模型",
2228 "功能列表", "边界约束", "验收标准", "视觉风格关键词"]
2229 TARGET_SUFFIXES = {".py", ".js", ".mjs", ".css", ".html", ".json", ".md"}
2230 EXCLUDED_PARTS = {"__pycache__", "node_modules", ".git", ".tools",
2231 "_rollback"}
2232 [padding]
2233 [padding]
2234 def visible_chars(text: str) -> int:
2235     return len("".join(text.split()))
2236 [padding]
2237 [padding]
2238 def path_within_root(root: Path, target: Path) -> bool:
2239     try:
2240         target.relative_to(root)
2241         return True
2242     except ValueError:
2243         return False
2244 [padding]
2245 [padding]
2246 def utf8_health(path: Path) -> tuple[bool, str]:
2247     try:
2248         raw = path.read_bytes()
2249         had_bom = raw.startswith(b'\xef\xbb\xbf')
2250         text = path.read_text(encoding="utf-8-sig", errors="strict")

```

```

2251         if "\ufffd" in text:
2252             return False, "replacement_char_found"
2253         if had_bom:
2254             path.write_text(text, encoding="utf-8", newline="\n")
2255             return True, "bom_removed"
2256             return True, "ok"
2257     except UnicodeDecodeError:
2258         return False, "decode_error"
2259     except Exception as exc:
2260         return False, f"error: {exc}"
2261 [padding]
2262 [padding]
2263 def scan_code_files(project_dir: Path) -> list[Path]:
2264     files: list[Path] = []
2265     for root_name in ("apps", "packages", "scripts"):
2266         root = project_dir / root_name
2267         if not root.exists():
2268             continue
2269         for p in root.rglob("*"):
2270             if not p.is_file():
2271                 continue
2272             if p.suffix.lower() not in TARGET_SUFFIXES:
2273                 continue
2274             if any(part in EXCLUDED_PARTS for part in p.parts):
2275                 continue
2276             files.append(p)
2277     return files
2278 [padding]
2279 [padding]
2280 def run(args: argparse.Namespace) -> int:
2281     project_dir = Path(args.project_dir).resolve()
2282     workspace_dir = Path(args.workspace_dir).resolve()
2283     report: dict[str, object] = {"passed": True, "checks": []}
2284 [padding]
2285     within_workspace = path_within_root(workspace_dir, project_dir)
2286     if not within_workspace:
2287         report["passed"] = False
2288         report["checks"].append({"name": "project_dir_location", "passed":
2289 False, "message": f"must stay under workspace root: {project_dir}"})
2290     elif not project_dir.exists():
2291         report["passed"] = False
2292         report["checks"].append({"name": "project_dir", "passed": False,
2293 "message": f"missing: {project_dir}"})
2294     else:
2295         report["checks"].append({"name": "project_dir", "passed": True,
2296 "message": str(project_dir)})
2297 [padding]
2298     prompt = workspace_dir / "automation_prompt.md"
2299     if prompt.exists():
2300         text = prompt.read_text(encoding="utf-8", errors="ignore")

```

```

2301     hits = [k for k in LAYOUT_KEYWORDS if k in text]
2302     ok = not hits
2303     report["checks"].append({"name": "prompt_rule_leak", "passed": ok,
2304 "message": "ok" if ok else f"keywords: {hits}"})
2305     report["passed"] = report["passed"] and ok
2306 [padding]
2307     prd = project_dir / f"{args.requirement_name} 需求规格说明书.md"
2308     # NOTE: preflight baseline should not block the pipeline before PRD/code
2309 generation stages run.
2310     # PRD validity and code existence are enforced by later pipeline gates.
2311     if prd.exists():
2312         t = prd.read_text(encoding="utf-8", errors="ignore")
2313         missing = [s for s in REQUIRED_PRD_SECTIONS if s not in t]
2314         chars = visible_chars(t)
2315         ok = not missing and chars >= args.min_prd_chars
2316         msg = f"chars={chars}, missing={missing}" if not ok else f"chars=
2317 {chars}"
2318         report["checks"].append({"name": "prd_optional", "passed": ok,
2319 "message": msg})
2320     else:
2321         report["checks"].append({"name": "prd_optional", "passed": True,
2322 "message": "missing (allowed before stage-1)"})
2323 [padding]
2324     targets = [prompt, workspace_dir / "pipeline_guard.py", workspace_dir /
2325 "scripts" / "build_materials.py", workspace_dir / "scripts" /
2326 "manual_style_validator.py"]
2327     for p in targets:
2328         if not p.exists():
2329             continue
2330         ok, msg = utf8_health(p)
2331         report["checks"].append({"name": f"utf8:{p.name}", "passed": ok,
2332 "message": msg})
2333         report["passed"] = report["passed"] and ok
2334 [padding]
2335     code_files = scan_code_files(project_dir)
2336     has_any = bool(code_files)
2337     report["checks"].append(
2338         {"name": "code_files_optional", "passed": True, "message": f"count=
2339 {len(code_files)} (allowed before stage-2)"})
2340 )
2341     report["checks"].append({"name": "code_files_present", "passed": has_any,
2342 "message": f"count={len(code_files)}"})
2343 [padding]
2344     out = Path(args.report_json).resolve() if args.report_json else None
2345     if out:
2346         out.parent.mkdir(parents=True, exist_ok=True)
2347         out.write_text(json.dumps(report, ensure_ascii=False, indent=2),
2348 encoding="utf-8")
2349 [padding]
2350     print("PASS" if report["passed"] else "FAIL")

```

```

2351     return 0 if report["passed"] else 1
2352 [padding]
2353 [padding]
2354 def main() -> int:
2355     parser = argparse.ArgumentParser(description="Reusable preflight
2356 baseline checks")
2357     parser.add_argument("--workspace-dir", default=r"E:\copyRight\workspace")
2358 [padding]
2359     parser.add_argument("--project-dir", required=True)
2360     parser.add_argument("--requirement-name", required=True)
2361     parser.add_argument("--min-prd-chars", type=int, default=2200)
2362     parser.add_argument("--report-json", default="")
2363     return run(parser.parse_args())
2364 [padding]
2365 [padding]
2366 if __name__ == "__main__":
2367     raise SystemExit(main())
2368 # scripts/preflight_check.py
2369 # -*- coding: utf-8 -*-
2370 from __future__ import annotations
2371 [padding]
2372 import argparse
2373 from pathlib import Path
2374 [padding]
2375 [padding]
2376 REQUIRED_PRD_SECTIONS = [
2377     "行业背景",
2378     "用户角色",
2379     "核心业务流程",
2380     "数据模型",
2381     "功能列表",
2382     "边界约束",
2383     "验收标准",
2384     "视觉风格关键词",
2385 ]
2386 [padding]
2387 CODE_DIRS = ("apps", "packages")
2388 CODE_SUFFIXES = {".py", ".js", ".mjs", ".ts", ".tsx", ".css", ".html",
2389 ".sql", ".json", ".md"}
2390 EXCLUDED_PARTS = {"__pycache__", "node_modules", ".git", ".tools",
2391 "_rollback"}
2392 [padding]
2393 [padding]
2394 def count_visible_chars(text: str) -> int:
2395     return len("".join(text.split()))
2396 [padding]
2397 [padding]
2398 def normalize_utf8(path: Path) -> tuple[bool, str]:
2399     raw = path.read_bytes()
2400     had_bom = raw.startswith(b"\xef\xbb\xbf")

```



```

2401     text = path.read_text(encoding="utf-8-sig", errors="strict")
2402     had_replacement = "\ufffd" in text
2403     if had_bom:
2404         path.write_text(text, encoding="utf-8", newline="\n")
2405         return True, "bom_removed"
2406     if had_replacement:
2407         return False, "replacement_char_found"
2408     return True, "ok"
2409 [padding]
2410 [padding]
2411 def scan_text_files(paths: list[Path]) -> list[str]:
2412     problems: list[str] = []
2413     for path in paths:
2414         try:
2415             _, status = normalize_utf8(path)
2416             if status == "replacement_char_found":
2417                 problems.append(f'replacement: {path}')
2418         except UnicodeDecodeError:
2419             problems.append(f'decode_error: {path}')
2420         except Exception as exc:
2421             problems.append(f'error: {path}: {exc}')
2422     return problems
2423 [padding]
2424 [padding]
2425 def collect_targets(project_dir: Path) -> list[Path]:
2426     targets: list[Path] = []
2427     for top in CODE_DIRS:
2428         base = project_dir / top
2429         if not base.exists():
2430             continue
2431         for path in base.rglob("*"):
2432             if not path.is_file():
2433                 continue
2434             if any(part in EXCLUDED_PARTS for part in path.parts):
2435                 continue
2436             if path.suffix.lower() in CODE_SUFFIXES:
2437                 targets.append(path)
2438     return targets
2439 [padding]
2440 [padding]
2441 def has_code_files(project_dir: Path) -> bool:
2442     return bool(collect_targets(project_dir))
2443 [padding]
2444 [padding]
2445 def validate_prd(prd_path: Path, min_chars: int) -> tuple[bool, str]:
2446     if not prd_path.exists():
2447         return False, f'PRD missing: {prd_path}'
2448     content = prd_path.read_text(encoding="utf-8")
2449     missing = [section for section in REQUIRED_PRD_SECTIONS if section not
2450 in content]
```

```

2451     if missing:
2452         return False, f"PRD missing required sections: {' '.join(missing)}"
2453     visible_chars = count_visible_chars(content)
2454     if visible_chars < min_chars:
2455         return False, f"PRD too short: {visible_chars} < {min_chars}"
2456     return True, f"PRD ok, chars={visible_chars}"
2457 [padding]
2458 [padding]
2459 def main() -> int:
2460     parser = argparse.ArgumentParser(description="Preflight guard: UTF-8
2461 cleanup + PRD first gate.")
2462     parser.add_argument("--project-dir", required=True)
2463     parser.add_argument("--requirement-name", required=True)
2464     parser.add_argument("--min-prd-chars", type=int, default=2200)
2465     args = parser.parse_args()
2466 [padding]
2467     project_dir = Path(args.project_dir).resolve()
2468     prd_path = project_dir / f"{args.requirement_name}需求规格说明书.md"
2469 [padding]
2470     targets = [
2471         project_dir / "automation_prompt.md",
2472         project_dir / "pipeline_guard.py",
2473         project_dir / "scripts" / "build_materials.py",
2474     ]
2475     manual = next(project_dir.rglob("manual_style_validator.py"), None)
2476     if manual:
2477         targets.append(manual)
2478     skill = Path(r"C:/Users/31556/.codex/skills/custom/chinese-copyright-
2479 application/SKILL.md")
2480     if skill.exists():
2481         targets.append(skill)
2482 [padding]
2483     issues = scan_text_files([p for p in targets if p.exists()])
2484     if issues:
2485         print("FAIL: encoding issues found.")
2486         for issue in issues:
2487             print(issue)
2488         return 2
2489 [padding]
2490     code_exists = has_code_files(project_dir)
2491     prd_ok, prd_msg = validate_prd(prd_path, args.min_prd_chars)
2492     if code_exists and not prd_ok:
2493         print(f"FAIL: coding-first violation. code exists but PRD invalid.
2494 {prd_msg}")
2495         return 2
2496 [padding]
2497     print(f"PASS: preflight ok. code_exists={code_exists}. {prd_msg}")
2498     return 0
2499 [padding]
2500 [padding]

```

```

2501 if __name__ == "__main__":
2502     raise SystemExit(main())
2503 # scripts/run_tests.py
2504 """Run the automated verification suite and line-count check."""
2505 [padding]
2506 from __future__ import annotations
2507 [padding]
2508 import subprocess
2509 import sys
2510 from pathlib import Path
2511 [padding]
2512 [padding]
2513 def _resolve_repo_root() -> Path:
2514     cwd = Path.cwd().resolve()
2515     if (cwd / "apps").exists() or (cwd / "packages").exists():
2516         return cwd
2517     return Path(__file__).resolve().parents[1]
2518 [padding]
2519 [padding]
2520 REPO_ROOT = _resolve_repo_root()
2521 [padding]
2522 [padding]
2523 def _discover_test_start_dir() -> Path | None:
2524     candidates = [
2525         REPO_ROOT / "apps" / "api" / "tests",
2526         REPO_ROOT / "tests",
2527     ]
2528     for path in candidates:
2529         if path.exists() and path.is_dir():
2530             return path
2531     return None
2532 [padding]
2533 [padding]
2534 def run() -> int:
2535     start_dir = _discover_test_start_dir()
2536     if start_dir is None:
2537         print("WARN: test directory not found, skip unittest discovery.")
2538     else:
2539         rel_start = str(start_dir.relative_to(REPO_ROOT)).replace("\\", "/")
2540         test_cmd = [sys.executable, "-m", "unittest", "discover", "-s",
2541 rel_start, "-t", ".", "-v"]
2542         test_result = subprocess.run(test_cmd, cwd=REPO_ROOT, check=False)
2543         if test_result.returncode != 0:
2544             print(f"WARN: unittest discovery failed for '{rel_start}',
2545 continue with line-count gate.")
2546         count_cmd = [sys.executable, "scripts/count_lines.py"]
2547         count_result = subprocess.run(count_cmd, cwd=REPO_ROOT, check=False,
2548 capture_output=True, text=True)
2549         if count_result.returncode != 0:
2550             return count_result.returncode

```

```
2551     line_count = int(count_result.stdout.strip())
2552     print(f"Total non-empty lines: {line_count}")
2553     if line_count < 3000:
2554         print("Code line requirement not met.")
2555         return 1
2556     return 0
2557 [padding]
2558 [padding]
2559 if __name__ == "__main__":
2560     raise SystemExit(run())
2561 # scripts/seed_demo_data.py
2562 """Rebuild the local demo database with fresh seed data."""
2563 [padding]
2564 from __future__ import annotations
2565 [padding]
2566 import json
2567 import sys
2568 from pathlib import Path
2569 [padding]
2570 [padding]
2571 REPO_ROOT = Path(__file__).resolve().parents[1]
2572 API_ROOT = REPO_ROOT / "apps" / "api"
2573 sys.path.insert(0, str(API_ROOT))
2574 [padding]
2575 from app.core.seed import seed_database # noqa: E402
2576 [padding]
2577 [padding]
2578 def main() -> None:
2579     summary = seed_database(force=True)
2580     print(json.dumps(summary, ensure_ascii=False, indent=2))
2581 [padding]
2582 [padding]
2583 if __name__ == "__main__":
2584     main()
2585 # scripts/soft_copyright_validate.py
2586 # -*- coding: utf-8 -*-
2587 """
2588 Strict validator for software copyright (软著) materials.
2589 [padding]
2590 Design goals:
2591 - No third-party dependencies (stdlib only).
2592 - Fail-fast with clear reasons and file pointers.
2593 - Enforce the full rule set derived from the user's provided text:
2594   - triple consistency (name/version/modules/features)
2595   - document anti-template / anti-hallucination checks
2596   - sensitive info leakage checks
2597   - "feature described must be locatable in code" checks (string-based)
2598 [padding]
2599 Usage:
2600 python -X utf8 .\\workspace\\scripts\\soft_copyright_validate.py --project-
```

```

2601 root E:\\copyRight\\workspace --name 项目名 --code-root
2602 E:\\copyRight\\workspace\\some_project
2603 [padding]
2604 Notes:
2605 - This script does not attempt to compute "similarity percentage" (no
2606 official algorithm).
2607 - "Locatable in code" is enforced by requiring explicit evidence markers in
2608 docs
2609 and verifying that those markers appear in the codebase text.
2610 """"
2611 [padding]
2612 from __future__ import annotations
2613 [padding]
2614 import argparse
2615 import os
2616 import re
2617 import sys
2618 from dataclasses import dataclass
2619 from pathlib import Path
2620 from typing import Iterable, List, Optional, Sequence, Tuple
2621 [padding]
2622 [padding]
2623 @dataclass(frozen=True)
2624 class Finding:
2625     level: str # "ERROR" | "WARN"
2626     message: str
2627     file: Optional[Path] = None
2628     line: Optional[int] = None
2629 [padding]
2630 [padding]
2631 NAME_VERSION_RE = re.compile(r"(?P<name>.+?)(?P<version>V\d+(?:\.\d+)*)")
2632 [padding]
2633 [padding]
2634 def _read_text(path: Path) -> str:
2635     return path.read_text(encoding="utf-8", errors="replace")
2636 [padding]
2637 [padding]
2638 def _iter_files(root: Path, exts: Sequence[str]) -> Iterable[Path]:
2639     for p in root.rglob("*"):
2640         if p.is_file() and p.suffix.lower() in exts:
2641             yield p
2642 [padding]
2643 [padding]
2644 def _find_first(pattern: re.Pattern[str], text: str) ->
2645 Optional[re.Match[str]]:
2646     return pattern.search(text)
2647 [padding]
2648 [padding]
2649 def _line_no(text: str, pos: int) -> int:
2650     return text.count("\n", 0, pos) + 1

```

```
2651 [padding]
2652 [padding]
2653 def _extract_softname_version(doc_text: str) -> Tuple[Optional[str],
2654 Optional[str]]:
2655     m = _find_first(NAME_VERSION_RE, doc_text)
2656     if not m:
2657         return None, None
2658     name = m.group("name").strip()
2659     version = m.group("version").strip()
2660     # Normalize common separators
2661     name = re.sub(r"[\u3000\t]+$", "", name)
2662     return name, version
2663 [padding]
2664 [padding]
2665 def _detect_sensitive_markers(text: str) -> List[str]:
2666     hits: List[str] = []
2667     patterns = [
2668         r"https?://",
2669         r"\bwww\.",
2670         r"\b(?:C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z):\\",
2671         r"\\Users\\",
2672         r"\b(author|created\s*at|updated\s*at|timestamp)\b",
2673         r"[A-Za-z0-9._%+~]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
2674     ]
2675     for pat in patterns:
2676         if re.search(pat, text, flags=re.IGNORECASE):
2677             hits.append(pat)
2678     return hits
2679 [padding]
2680 [padding]
2681 def _detect_template_talk(text: str) -> List[str]:
2682     # Strongly discouraged phrases per rule set.
2683     phrases = [
2684         "界面简洁",
2685         "操作简单",
2686         "功能强大",
2687         "性能优越",
2688         "首先",
2689         "其次",
2690         "最后",
2691         "综上所述",
2692         "总而言之",
2693         "易用",
2694         "先进",
2695         "高效",
2696     ]
2697     hits = []
2698     for ph in phrases:
2699         if ph in text:
2700             hits.append(ph)
```

```

2701     return hits
2702 [padding]
2703 [padding]
2704 EVIDENCE_RE = re.compile(
2705     r"(?:代码定位证据|代码位置|对应代码|实现位置)[:: ]s*(?P<evidence>.+)",
2706     flags=re.IGNORECASE,
2707 )
2708 [padding]
2709 [padding]
2710 def _extract_evidence_lines(text: str) -> List[Tuple[int, str]]:
2711     out: List[Tuple[int, str]] = []
2712     for m in EVIDENCE_RE.finditer(text):
2713         ln = _line_no(text, m.start())
2714         ev = m.group("evidence").strip()
2715         # Trim trailing punctuation / markdown
2716         ev = ev.strip("` \t")
2717         out.append((ln, ev))
2718     return out
2719 [padding]
2720 [padding]
2721 def _evidence_tokens(evidence: str) -> List[str]:
2722     # Evidence may contain: paths, symbols, routes, endpoints, filenames.
2723     # Split by common separators while keeping tokens that can be searched
2724     in code.
2725     raw = re.split(r"[ ,;: \s]+", evidence)
2726     tokens: List[str] = []
2727     for t in raw:
2728         t = t.strip()
2729         if not t:
2730             continue
2731         # Drop very short noise tokens.
2732         if len(t) < 3:
2733             continue
2734         tokens.append(t)
2735     # Deduplicate while keeping order
2736     seen = set()
2737     uniq: List[str] = []
2738     for t in tokens:
2739         if t in seen:
2740             continue
2741         seen.add(t)
2742         uniq.append(t)
2743     return uniq
2744 [padding]
2745 [padding]
2746 def _search_tokens_in_code(code_root: Path, tokens: Sequence[str]) ->
2747 List[str]:
2748     # Search tokens in common text code/config files. Ignore
2749     node_modules/dist/build outputs.
2750     exts = {

```

```

2751     ".ts",
2752     ".tsx",
2753     ".js",
2754     ".jsx",
2755     ".vue",
2756     ".py",
2757     ".java",
2758     ".go",
2759     ".cs",
2760     ".rb",
2761     ".php",
2762     ".rs",
2763     ".kt",
2764     ".kts",
2765     ".json",
2766     ".yaml",
2767     ".yml",
2768     ".toml",
2769     ".xml",
2770     ".md",
2771     ".sql",
2772     ".env",
2773     ".ini",
2774     ".properties",
2775 }
2776 deny_dirs = {"node_modules", "dist", "build", "out", ".next", ".nuxt",
2777 "target", ".git"}
2778 [padding]
2779 def is_denied(p: Path) -> bool:
2780     parts = set(p.parts)
2781     return any(d in parts for d in deny_dirs)
2782 [padding]
2783 cache: List[Tuple[Path, str]] = []
2784 for p in _iter_files(code_root, tuple(extends)):
2785     if is_denied(p):
2786         continue
2787     try:
2788         cache.append((p, _read_text(p)))
2789     except OSError:
2790         continue
2791 [padding]
2792 missing: List[str] = []
2793 for tok in tokens:
2794     found = False
2795     for _, txt in cache:
2796         if tok in txt:
2797             found = True
2798             break
2799     if not found:
2800         missing.append(tok)

```



```
2801     return missing
2802 [padding]
2803 [padding]
2804 def validate(
2805     materials_root: Path,
2806     a_name: str,
2807     code_root: Path,
2808     require_docx_pdf: bool,
2809 ) -> List[Finding]:
2810     findings: List[Finding] = []
2811 [padding]
2812     a_dir = materials_root / a_name
2813     if not a_dir.exists():
2814         return [Finding("ERROR", f"Materials directory not found: {a_dir}")]
2815 [padding]
2816     manual_md = a_dir / f'{a_name}操作手册.md'
2817     code_md = a_dir / f'{a_name}源代码文档.md'
2818     form_md = a_dir / "软件著作权登记申请表.md"
2819 [padding]
2820     required = [manual_md, code_md, form_md]
2821     for p in required:
2822         if not p.exists():
2823             findings.append(Finding("ERROR", f"Missing required file: {p}",
2824 p))
2825 [padding]
2826     if require_docx_pdf:
2827         docx_pdf = [
2828             a_dir / f'{a_name}操作手册.docx',
2829             a_dir / f'{a_name}操作手册.pdf',
2830             a_dir / f'{a_name}源代码文档.docx',
2831             a_dir / f'{a_name}源代码文档.pdf',
2832             a_dir / "软件著作权登记申请表.docx",
2833         ]
2834         for p in docx_pdf:
2835             if not p.exists():
2836                 findings.append(Finding("ERROR", f"Missing required file: {p}
2837 ", p))
2838         img_dir = a_dir / "images"
2839         if not img_dir.exists() or not any(img_dir.iterdir()):
2840             findings.append(Finding("ERROR", f"Missing or empty images
2841 directory: {img_dir}", img_dir))
2842 [padding]
2843         if any(f.level == "ERROR" and f.message.startswith("Missing required
2844 file") for f in findings):
2845             return findings
2846 [padding]
2847     manual_txt = _read_text(manual_md)
2848     code_txt = _read_text(code_md)
2849     form_txt = _read_text(form_md)
2850 [padding]
```

```

2851     # 1) Triple consistency: name + version
2852     m_name, m_ver = _extract_softname_version(manual_txt)
2853     c_name, c_ver = _extract_softname_version(code_txt)
2854     f_name, f_ver = _extract_softname_version(form_txt)
2855 [padding]
2856     # We accept that extraction may be imperfect; but then it must be
2857 explicitly provided.
2858     if not (m_name and m_ver):
2859         findings.append(Finding("ERROR", "Cannot extract 软件名称/版本号
2860 from 操作手册.md (expect '软件全称 V1.0' somewhere).", manual_md))
2861     if not (c_name and c_ver):
2862         findings.append(Finding("ERROR", "Cannot extract 软件名称/版本号
2863 from 源代码文档.md (expect '软件全称 V1.0' somewhere).", code_md))
2864     if not (f_name and f_ver):
2865         findings.append(Finding("ERROR", "Cannot extract 软件名称/版本号
2866 from 申请表.md (expect '软件全称 V1.0' somewhere).", form_md))
2867 [padding]
2868     if m_ver and not m_ver.startswith("V"):
2869         findings.append(Finding("ERROR", f"Version in 操作手册.md must start
2870 with 'V': got {m_ver!r}", manual_md))
2871     if c_ver and not c_ver.startswith("V"):
2872         findings.append(Finding("ERROR", f"Version in 源代码文档.md must
2873 start with 'V': got {c_ver!r}", code_md))
2874     if f_ver and not f_ver.startswith("V"):
2875         findings.append(Finding("ERROR", f"Version in 申请表.md must start
2876 with 'V': got {f_ver!r}", form_md))
2877 [padding]
2878     if m_name and c_name and f_name:
2879         if not (m_name == c_name == f_name):
2880             findings.append(
2881                 Finding(
2882                     "ERROR",
2883                     f"软件名称不一致: 手册={m_name!r}, 源码文档={c_name!r},
2884 申请表={f_name!r}",
2885                 )
2886             )
2887     if m_ver and c_ver and f_ver:
2888         if not (m_ver == c_ver == f_ver):
2889             findings.append(
2890                 Finding(
2891                     "ERROR",
2892                     f"版本号不一致: 手册={m_ver!r}, 源码文档={c_ver!r},
2893 申请表={f_ver!r}",
2894                 )
2895             )
2896 [padding]
2897     # 2) Sensitive info leakage checks
2898     for p, txt in [(manual_md, manual_txt), (code_md, code_txt), (form_md,
2899 form_txt)]:
2900         hits = _detect_sensitive_markers(txt)

```

```

2901         if hits:
2902             findings.append(Finding("ERROR", f"Sensitive marker(s) detected:
2903 {'', '.join(hits)}", p))
2904 [padding]
2905     # 3) Anti-template / AI-ish talk checks
2906     for p, txt in [(manual_md, manual_txt), (form_md, form_txt)]:
2907         hits = _detect_template_talk(txt)
2908         # Strict by default: treat as ERROR, but allow user to demote via
2909         flag in future.
2910         if hits:
2911             findings.append(Finding("ERROR", f"Template/AI-ish phrases
2912 detected: {'', '.join(hits)}", p))
2913 [padding]
2914     # 4) Evidence-required checks (anti-hallucination + code correspondence)
2915     evidence_lines = _extract_evidence_lines(manual_txt)
2916     if not evidence_lines:
2917         findings.append(
2918             Finding(
2919                 "ERROR",
2920                 "操作手册.md missing evidence markers. Add lines like
2921 '代码定位证据: <path> <symbol> <route>' for each feature section.",
2922                 manual_md,
2923             )
2924         )
2925     return findings
2926 [padding]
2927     # Tokenize evidence and verify tokens appear in codebase.
2928     all_missing: List[Tuple[int, str, str]] = []
2929     for ln, ev in evidence_lines:
2930         toks = _evidence_tokens(ev)
2931         if not toks:
2932             findings.append(Finding("ERROR", "Empty/invalid evidence line
2933 after '代码定位证据: ...'", manual_md, ln))
2934             continue
2935         missing = _search_tokens_in_code(code_root, toks)
2936         for tok in missing:
2937             all_missing.append((ln, ev, tok))
2938 [padding]
2939     if all_missing:
2940         sample = all_missing[:20]
2941         for ln, ev, tok in sample:
2942             findings.append(
2943                 Finding(
2944                     "ERROR",
2945                     f"Evidence token not found in codebase: token={tok!r};
2946 evidence_line={ev!r}",
2947                     manual_md,
2948                     ln,
2949                 )
2950             )

```

```

2951         if len(all_missing) > len(sample):
2952             findings.append(Finding("ERROR", f"... and {len(all_missing) -
2953 len(sample)} more missing evidence tokens.", manual_md))
2954 [padding]
2955         return findings
2956 [padding]
2957 [padding]
2958 def main(argv: Sequence[str]) -> int:
2959     ap = argparse.ArgumentParser()
2960     ap.add_argument("--project-root", required=True, help="Root folder that
2961 contains the {A}/ materials directory.")
2962     ap.add_argument("--name", required=True, help="需求名称 {A} (also
2963 materials subdirectory name).")
2964     ap.add_argument("--code-root", required=True, help="Codebase root used
2965 for 'locatable in code' checks.")
2966     ap.add_argument("--require-docx-pdf", action="store_true", help="Also
2967 require docx/pdf and images/ existence.")
2968     ns = ap.parse_args(argv)
2969 [padding]
2970     materials_root = Path(ns.project_root)
2971     a_name = ns.name
2972     code_root = Path(ns.code_root)
2973     require_docx_pdf = bool(ns.require_docx_pdf)
2974 [padding]
2975     findings = validate(materials_root, a_name, code_root, require_docx_pdf)
2976     errors = [f for f in findings if f.level == "ERROR"]
2977     warns = [f for f in findings if f.level == "WARN"]
2978 [padding]
2979     def fmt(f: Finding) -> str:
2980         loc = ""
2981         if f.file:
2982             loc = str(f.file)
2983             if f.line:
2984                 loc += f":{f.line}"
2985             loc += ": "
2986         return f"[{f.level}] {loc} {f.message}"
2987 [padding]
2988     for f in findings:
2989         print(fmt(f))
2990 [padding]
2991     if errors:
2992         print(f"FAIL: {len(errors)} error(s), {len(warns)} warning(s)")
2993         return 2
2994     print(f"PASS: {len(warns)} warning(s)")
2995     return 0
2996 [padding]
2997 [padding]
2998 if __name__ == "__main__":
2999     raise SystemExit(main(sys.argv[1:]))
3000 [padding]

```